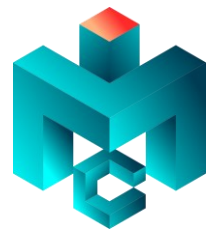




UNIVERSIDADE FEDERAL DE ITAJUBÁ
Instituto de Matemática e Computação



CMAC03 – Algoritmos em Grafos

2. Conceitos de Teoria dos Grafos

Percurso e Conectividade

Rafael Frinhani

frinhani@unifei.edu.br

2026

Apresentar as definições e tipos de percursos em grafos não-direcionados e direcionados, circuito Euleriano e ciclo Hamiltoniano.
Conceitos e definições de conectividade em grafos.

AGENDA

2. Teoria dos Grafos

2.5. Estruturas de Percurso

Contextualização, Definições (Passeio, Trajeto, Circuito, Caminho, Ciclo), Igualdade de Ciclos, Grafo Acíclico Direcionado, Identificação do Percurso, Fecho Transitivo (Direto e Indireto), Alcançabilidade. Circuito Euleriano, Ciclo Hamiltoniano, aplicações.

2.6. Conectividade

Definição de Conectividade (grafo conexo e desconexo) em grafos não-direcionados e em grafos direcionados (s-conexo, sf-conexo, f-conexo). Conectividade em Vértices, k -conectividade. Articulação (Vértice, Arestas). Cliques.



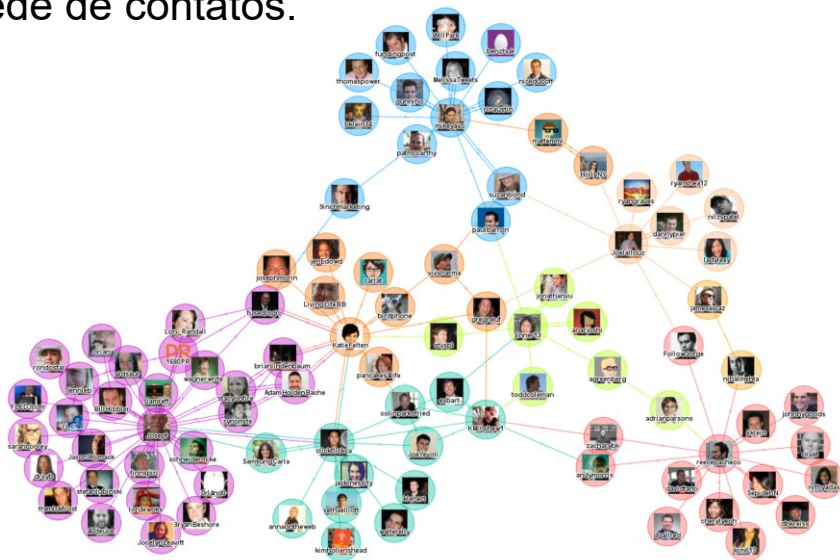


2.5. Estruturas de Percurso

Contextualização

Identificar um percurso em um grafo é fundamental na determinação das possibilidades de interações entre componentes de um sistema nos mais variados contextos.

Percursos são úteis na **verificação de conexões** entre indivíduos (ex. redes sociais, acadêmicas, criminais). **Ex.** ligação entre indivíduos desconhecidos considerando sua rede de contatos.



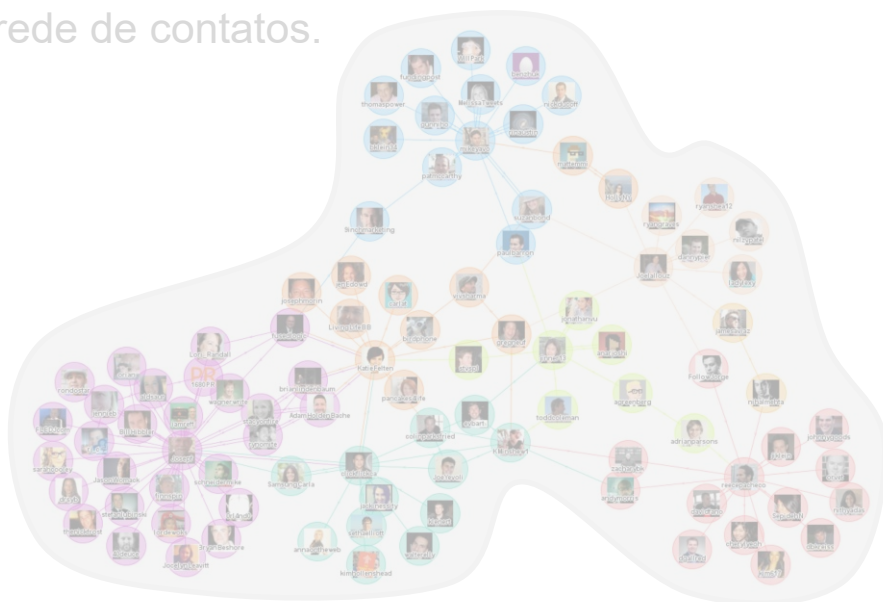


2.5. Estruturas de Percurso

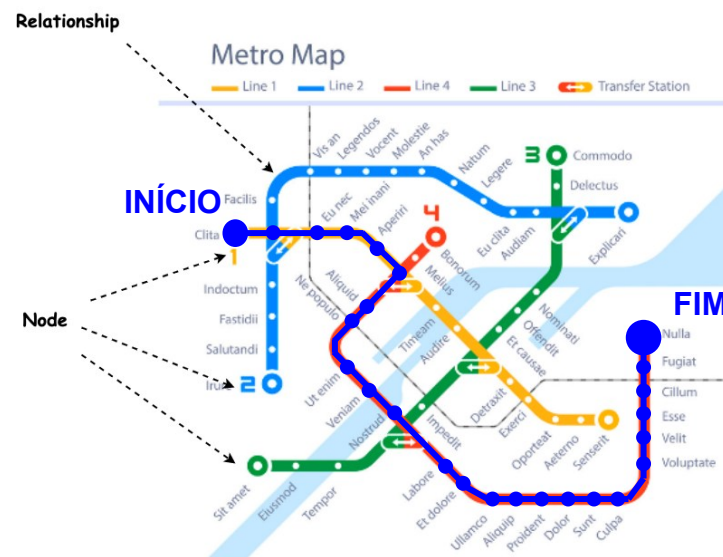
Contextualização

Identificar um percurso em um grafo é fundamental na determinação das possibilidades de interações entre componentes de um sistema nos mais variados contextos.

Caminhos são úteis na verificação de conexões entre indivíduos (ex. redes sociais, acadêmicas, criminais). **Ex.** ligação entre indivíduos desconhecidos considerando sua rede de contatos.



Um caminho permite **verificar se um local destino pode ser alcançado** a partir de uma origem. **Ex.** Planejamento de rotas de serviços de entrega, coleta de lixo, etc.





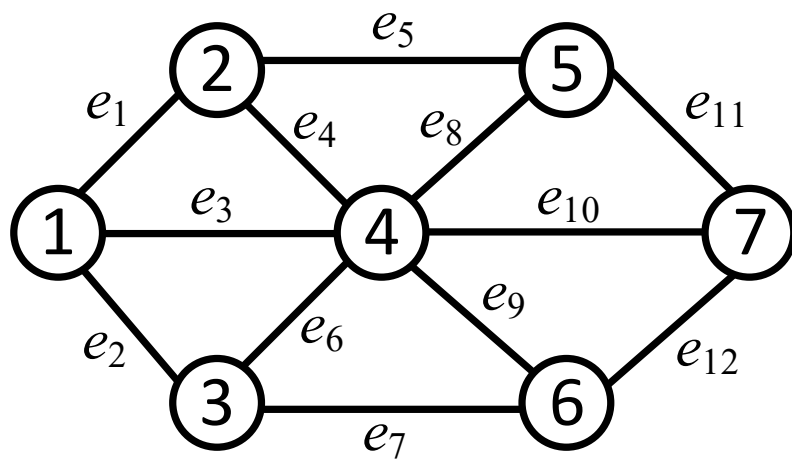
2.5. Estruturas de Percurso

Passeio

Um passeio (*walk*) em um grafo $G = (V, E)$ é uma sequência $W = v_0 e_1 v_1 \dots v_{k-1} e_k v_k$ cujos termos são alternadamente **vértices** e **arestas** de G , **distintos ou não**, tais que v_{i-1} e v_i são vértices terminais de e_i , $1 \leq i \leq k$.

Se $v_0 = x$ e $v_k = y$ é dito que W conecta x a y sendo W referido como um passeio xy .

No passeio é **permitido repetir** vértices e arestas.



Comprimento do passeio é sua **quantidade de arestas**. Um passeio com n vértices possui $n - 1$ arestas.



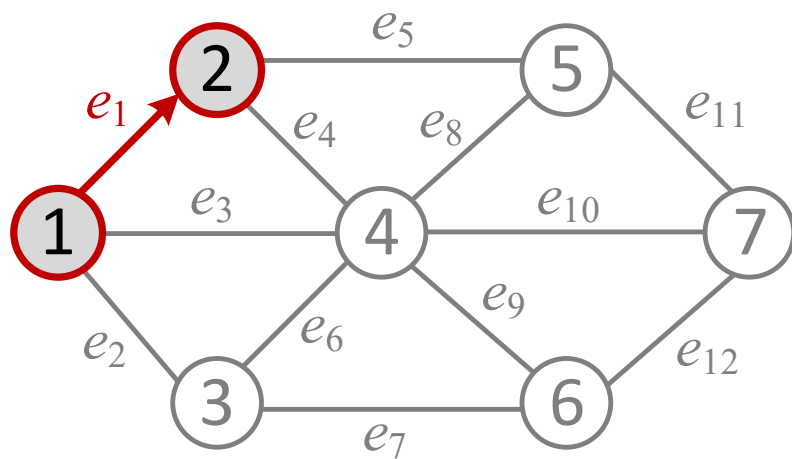
2.5. Estruturas de Percurso

Passeio

Um passeio (*walk*) em um grafo $G = (V, E)$ é uma sequência $W = v_0 e_1 v_1 \dots v_{k-1} e_k v_k$ cujos termos são alternadamente vértices e arestas de G , distintos ou não, tais que v_{i-1} e v_i são vértices terminais de e_i , $1 \leq i \leq k$.

Se $v_0 = x$ e $v_k = y$ é dito que W conecta x a y sendo W referido como um passeio xy .

No passeio é permitido repetir vértices e arestas.



Comprimento do passeio é sua quantidade de arestas. Um passeio com n vértices possui $n - 1$ arestas.

Exemplo:

$$W = 1e_12$$



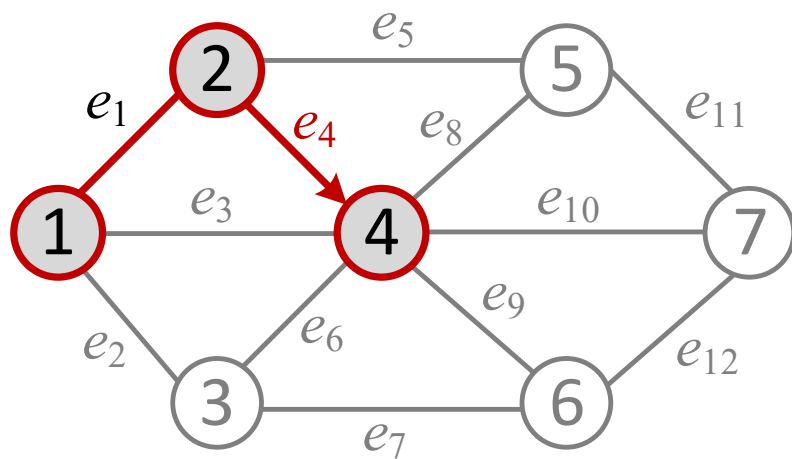
2.5. Estruturas de Percurso

Passeio

Um passeio (*walk*) em um grafo $G = (V, E)$ é uma sequência $W = v_0 e_1 v_1 \dots v_{k-1} e_k v_k$ cujos termos são alternadamente vértices e arestas de G , distintos ou não, tais que v_{i-1} e v_i são vértices terminais de e_i , $1 \leq i \leq k$.

Se $v_0 = x$ e $v_k = y$ é dito que W conecta x a y sendo W referido como um passeio xy .

No passeio é permitido repetir vértices e arestas.



Comprimento do passeio é sua quantidade de arestas. Um passeio com n vértices possui $n - 1$ arestas.

Exemplo:

$$W = 1e_12e_44$$



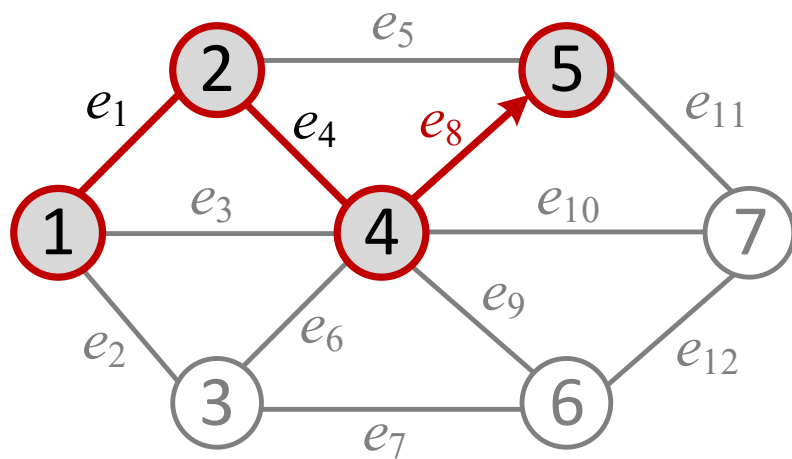
2.5. Estruturas de Percurso

Passeio

Um passeio (*walk*) em um grafo $G = (V, E)$ é uma sequência $W = v_0 e_1 v_1 \dots v_{k-1} e_k v_k$ cujos termos são alternadamente vértices e arestas de G , distintos ou não, tais que v_{i-1} e v_i são vértices terminais de e_i , $1 \leq i \leq k$.

Se $v_0 = x$ e $v_k = y$ é dito que W conecta x a y sendo W referido como um passeio xy .

No passeio é permitido repetir vértices e arestas.



Comprimento do passeio é sua quantidade de arestas. Um passeio com n vértices possui $n - 1$ arestas.

Exemplo:

$$W = 1e_12e_44e_85$$



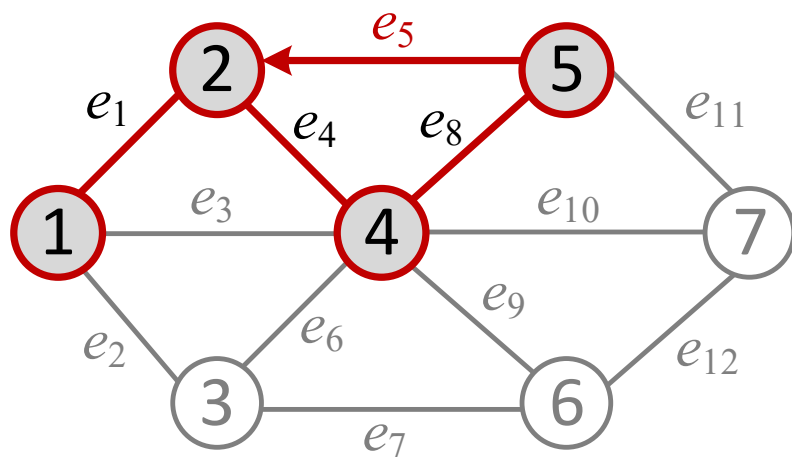
2.5. Estruturas de Percurso

Passeio

Um passeio (*walk*) em um grafo $G = (V, E)$ é uma sequência $W = v_0 e_1 v_1 \dots v_{k-1} e_k v_k$ cujos termos são alternadamente vértices e arestas de G , distintos ou não, tais que v_{i-1} e v_i são vértices terminais de e_i , $1 \leq i \leq k$.

Se $v_0 = x$ e $v_k = y$ é dito que W conecta x a y sendo W referido como um passeio xy .

No passeio é permitido repetir vértices e arestas.



Comprimento do passeio é sua quantidade de arestas. Um passeio com n vértices possui $n - 1$ arestas.

Exemplo:

$$W = 1e_12e_44e_85e_52$$



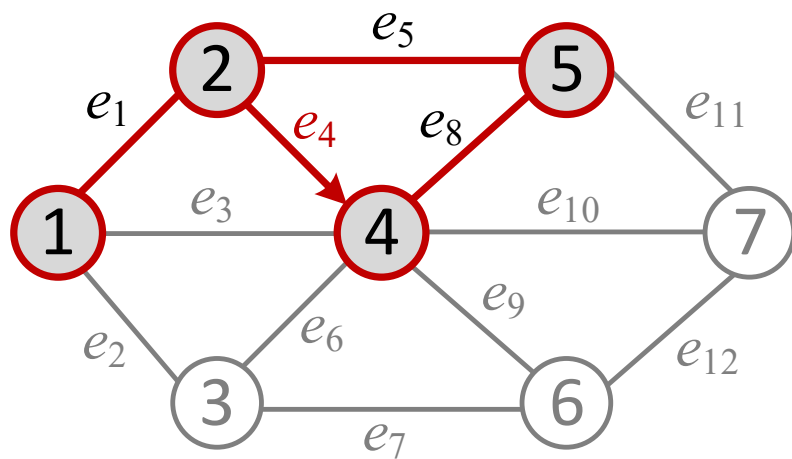
2.5. Estruturas de Percurso

Passeio

Um passeio (*walk*) em um grafo $G = (V, E)$ é uma sequência $W = v_0 e_1 v_1 \dots v_{k-1} e_k v_k$ cujos termos são alternadamente vértices e arestas de G , distintos ou não, tais que v_{i-1} e v_i são vértices terminais de e_i , $1 \leq i \leq k$.

Se $v_0 = x$ e $v_k = y$ é dito que W conecta x a y sendo W referido como um passeio xy .

No passeio é permitido repetir vértices e arestas.



Comprimento do passeio é sua quantidade de arestas. Um passeio com n vértices possui $n - 1$ arestas.

Exemplo:

$$W = 1e_12e_44e_85e_52e_44$$



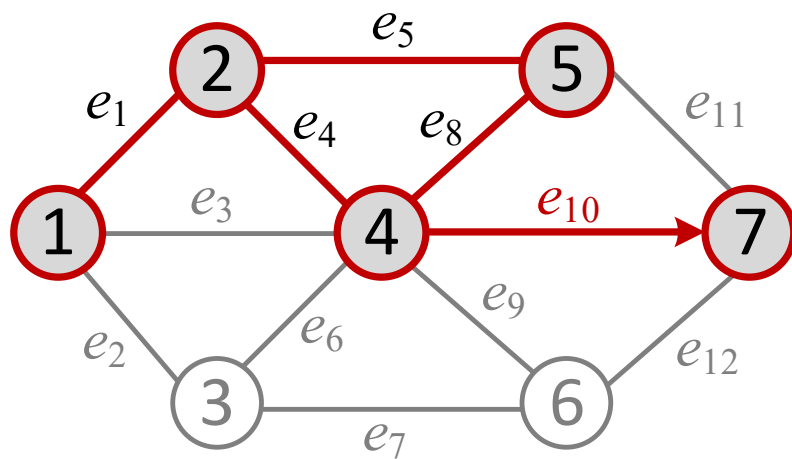
2.5. Estruturas de Percurso

Passeio

Um passeio (*walk*) em um grafo $G = (V, E)$ é uma sequência $W = v_0 e_1 v_1 \dots v_{k-1} e_k v_k$ cujos termos são alternadamente vértices e arestas de G , distintos ou não, tais que v_{i-1} e v_i são vértices terminais de e_i , $1 \leq i \leq k$.

Se $v_0 = x$ e $v_k = y$ é dito que W conecta x a y sendo W referido como um passeio xy .

No passeio é permitido repetir vértices e arestas.



Comprimento do passeio é sua quantidade de arestas. Um passeio com n vértices possui $n - 1$ arestas.

Exemplo:

$$W = 1e_12e_44e_85e_52e_44e_{10}7$$



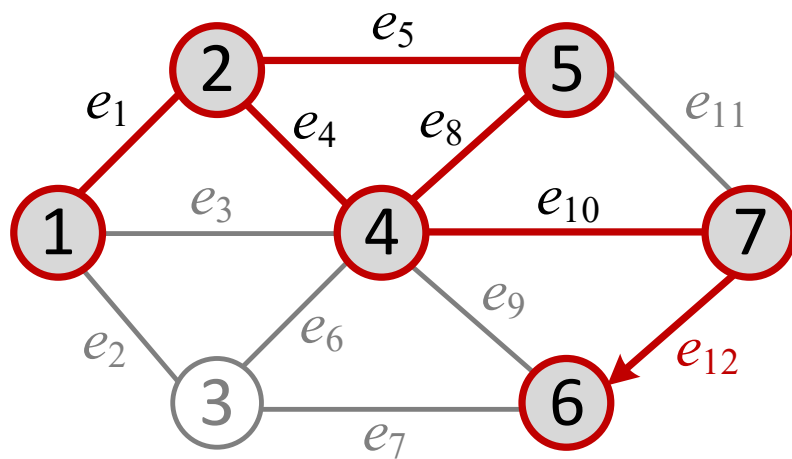
2.5. Estruturas de Percurso

Passeio

Um passeio (*walk*) em um grafo $G = (V, E)$ é uma sequência $W = v_0 e_1 v_1 \dots v_{k-1} e_k v_k$ cujos termos são alternadamente vértices e arestas de G , distintos ou não, tais que v_{i-1} e v_i são vértices terminais de e_i , $1 \leq i \leq k$.

Se $v_0 = x$ e $v_k = y$ é dito que W conecta x a y sendo W referido como um passeio xy .

No passeio é permitido repetir vértices e arestas.



Comprimento do passeio é sua quantidade de arestas. Um passeio com n vértices possui $n - 1$ arestas.

Exemplo:

$$W = 1e_12e_44e_85e_52e_44e_{10}7e_{12}6$$

comprimento = **7**

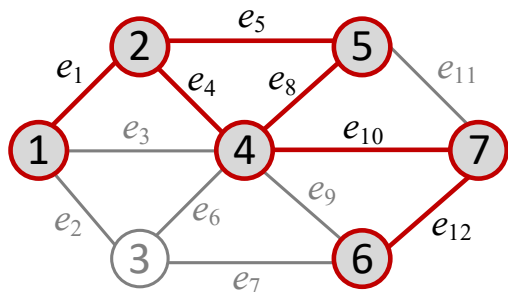


2.5. Estruturas de Percurso

Passeio

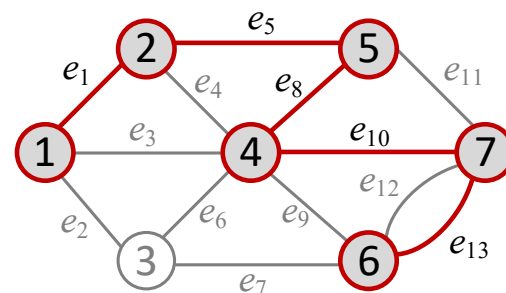
- **Grafo simples:** o passeio pode ser representado apenas pelos vértices.

$W = 1\ 2\ 4\ 5\ 2\ 4\ 7\ 6$



- **Multigrafos e Pseudografos:** deve-se indicar qual aresta é usada:

$W = 1e_12e_55e_84e_{10}7e_{13}6$



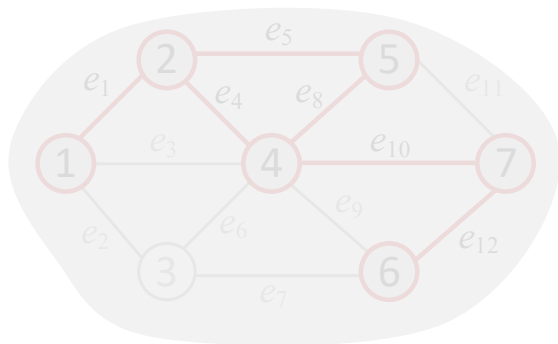


2.5. Estruturas de Percurso

Passeio

- Grafo simples: o passeio pode ser representado apenas pelos vértices.

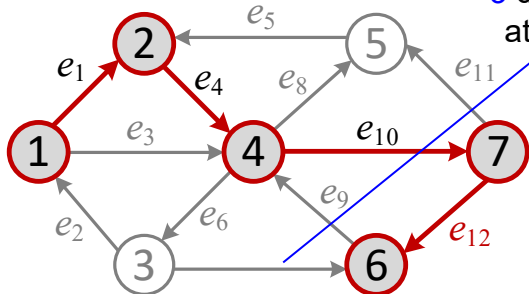
$W = 1\ 2\ 4\ 5\ 2\ 4\ 7\ 6$



- Digrafo:** deve-se respeitar o sentido dos arcos.

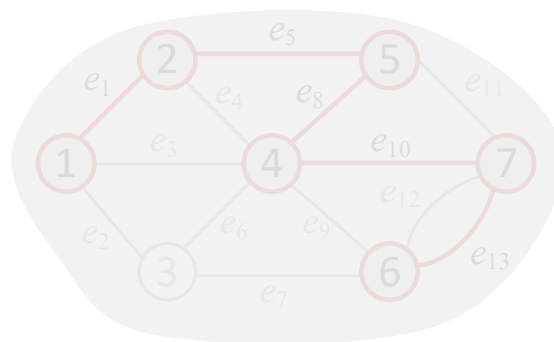
$W = 1\ 2\ 4\ 7\ 6$

Não é possível ir de 6 para 3 diretamente, apenas através do vértice 4.



- Multigrafos e Pseudografos: deve-se indicar qual aresta é usada:

$W = 1e_12e_55e_84e_{10}7e_{13}6$



- O **caminho trivial** de v para v consiste apenas do vértice v .
- Se existir um passeio W de x para y então é dito que y é **alcançável** a partir de x via W .



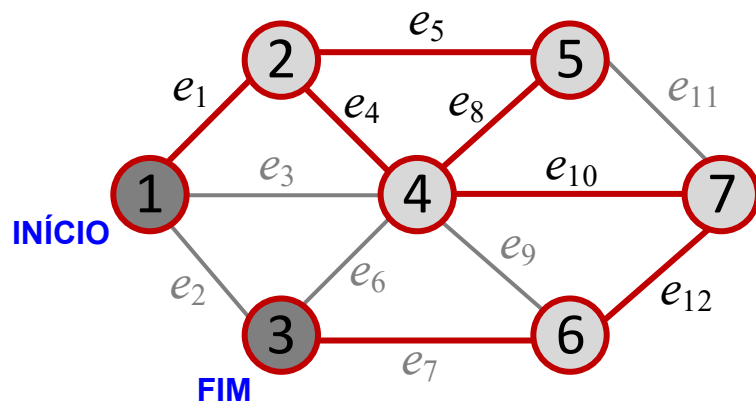
2.5. Estruturas de Percurso

Passeio

Os vértices x (**inicial**) e y (**terminal**) são chamados **extremidades do passeio**, e os vértices $v_1, \dots, v_{k-1}$ denominados vértices internos.

Passeio Aberto: Quando os **vértices inicial** e **final** são diferentes.

$$W = 1e_12e_44e_85e_52e_44e_{10}7e_{12}6e_73$$





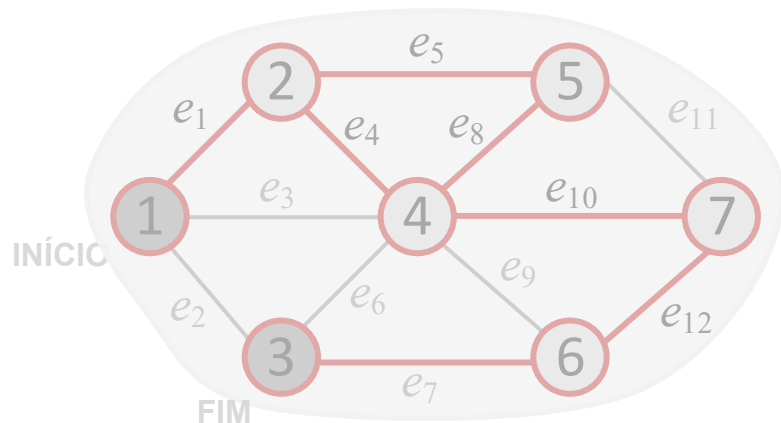
2.5. Estruturas de Percurso

Passeio

Os vértices x (inicial) e y (terminal) são chamados extremidades do passeio, e os vértices $v_1, \dots, v_{k-1}$ denominados vértices internos.

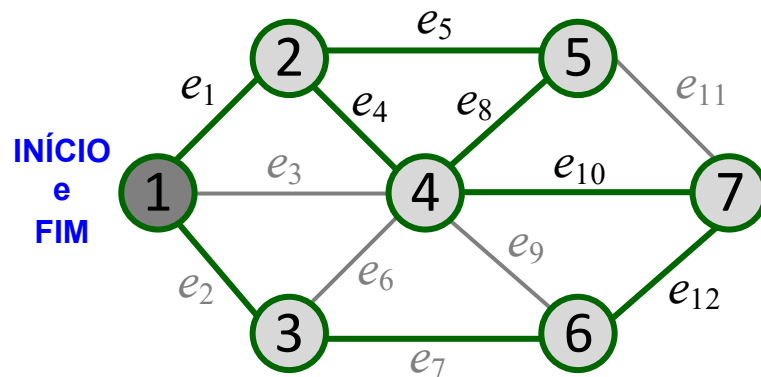
Passeio Aberto: Quando os vértices inicial e final são diferentes.

$$W = 1e_12e_44e_85e_52e_44e_{10}7e_{12}6e_73$$



Passeio Fechado: Quando os vértices inicial e final são iguais.

$$W = 1e_12e_44e_85e_52e_44e_{10}7e_{12}6e_73e_21$$





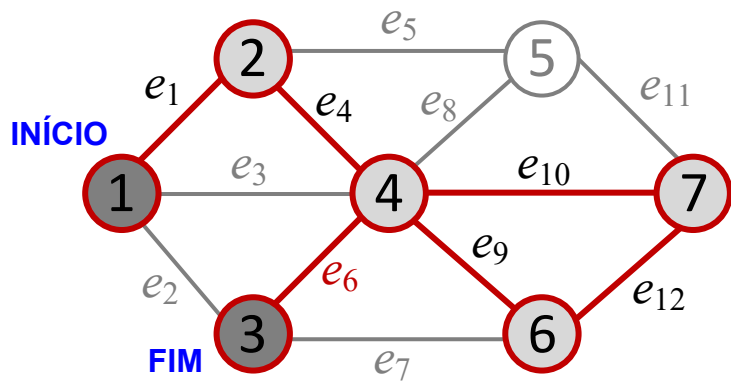
2.5. Estruturas de Percurso

Trajeto

Um trajeto (trilha, *trail*) em um grafo $G = (V, E)$ é uma sequência $T = v_0 e_1 v_1 \dots v_{k-1} e_k v_k$ cujos termos são alternadamente vértices e arestas de G , sendo que todas as arestas são distintas, tais que v_{i-1} e v_i são vértices terminais de e_i , $1 \leq i \leq k$.

É permitido repetir vértices mas não é permitido repetir arestas.

Trajeto Aberto: Tem início e término em vértices diferentes.



$$W = 1e_12e_44e_{10}7e_{12}6e_94e_63$$



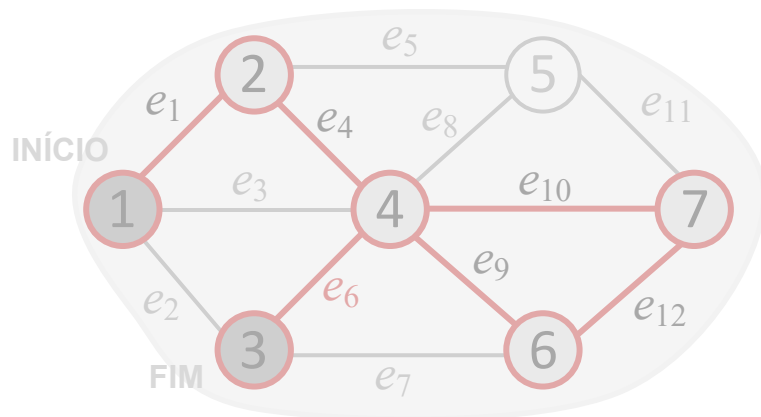
2.5. Estruturas de Percurso

Trajeto

Um trajeto (trilha, *trail*) em um grafo $G = (V, E)$ é uma sequência $T = v_0 e_1 v_1 \dots v_{k-1} e_k v_k$ cujos termos são alternadamente vértices e arestas de G , sendo que todas as arestas são distintas, tais que v_{i-1} e v_i são vértices terminais de e_i , $1 \leq i \leq k$.

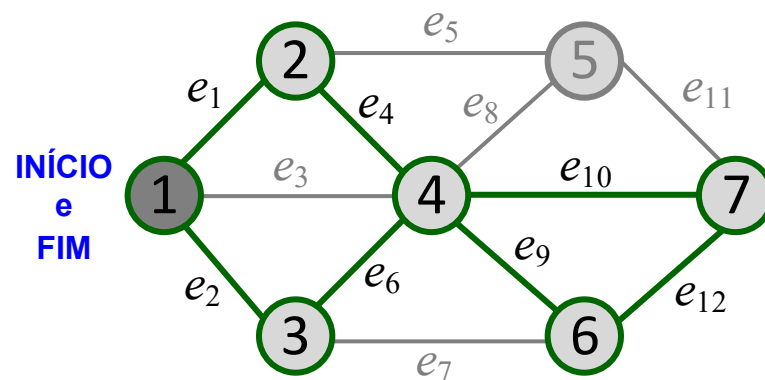
É permitido repetir vértices mas não é permitido repetir arestas.

Trajeto Aberto: Tem início e término em vértices diferentes.



$$W = 1e_12e_44e_{10}7e_{12}6e_94e_63$$

Trajeto Fechado (Circuito): Tem início e término no mesmo vértice.



$$W = 1e_12e_44e_{10}7e_{12}6e_94e_63e_21$$



2.5. Estruturas de Percurso

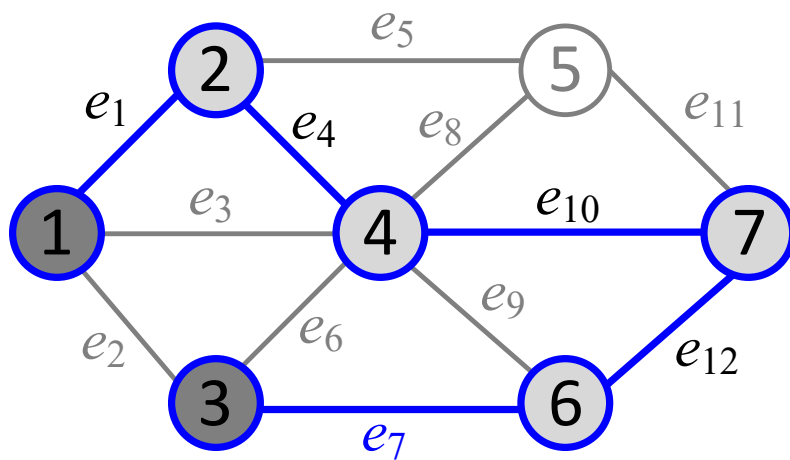
Caminho

Um caminho (*path*) em um grafo $G = (V, E)$ é uma sequência $P = v_0 e_1 v_1 \dots v_{k-1} e_k v_k$ cujos termos são alternadamente vértices e arestas de G , sendo que todos os vértices e todas as arestas são distintas, tais que v_{i-1} e v_i são vértices terminais de e_i , $1 \leq i \leq k$.

Não é permitido **repetir vértices e arestas**. O caminho é aberto ($v_{i-1} \neq v_i$).

Exemplo:

$$W = 1e_12e_44e_{10}7e_{12}6e_73$$





2.5. Estruturas de Percurso

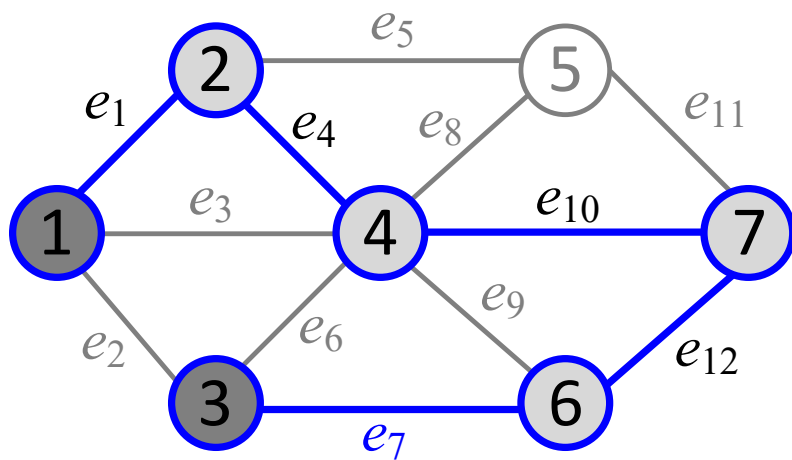
Caminho

Um caminho (*path*) em um grafo $G = (V, E)$ é uma sequência $P = v_0 e_1 v_1 \dots v_{k-1} e_k v_k$ cujos termos são alternadamente vértices e arestas de G , sendo que todos os vértices e todas as arestas são distintas, tais que v_{i-1} e v_i são vértices terminais de e_i , $1 \leq i \leq k$.

Não é permitido repetir vértices e arestas. O caminho é aberto ($v_{i-1} \neq v_i$).

Exemplo:

$W = 1e_12e_44e_{10}7e_{12}6e_73$



Algoritmos para **obter caminhos** são baseados nos algoritmos de **busca em grafos** (ex. DFS, BFS etc).

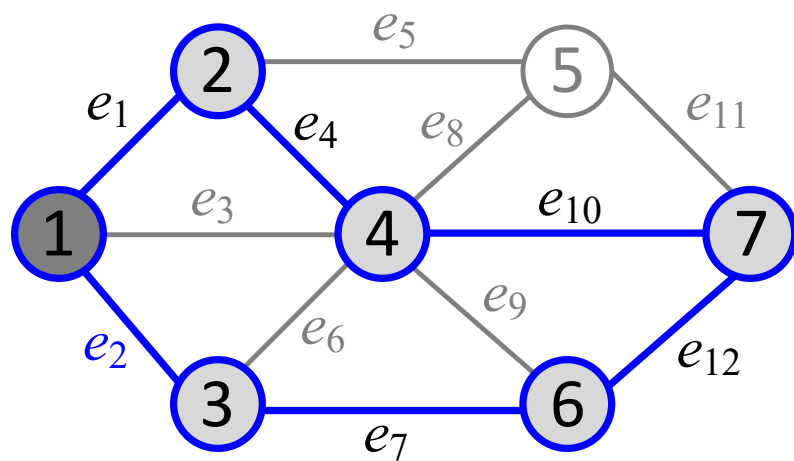
A estratégia básica é explorar rotas entre vértices percorrendo os relacionamentos até que se alcance o destino.



2.5. Estruturas de Percurso

Ciclo

Um ciclo (**caminho fechado**, *cycle*) em um grafo $G = (V, E)$ é uma sequência $C = v_0 e_1 v_1 \dots v_{k-1} e_k v_0$ cujos termos são alternadamente vértices e arestas de G , sendo que **todos os vértices e todas as arestas são distintas**, tais que v_{i-1} e v_i são vértices terminais de e_i , $1 \leq i \leq k$. **Apenas** vértices **inicial** e **final** são **idênticos** ($v_0 = v_k$).



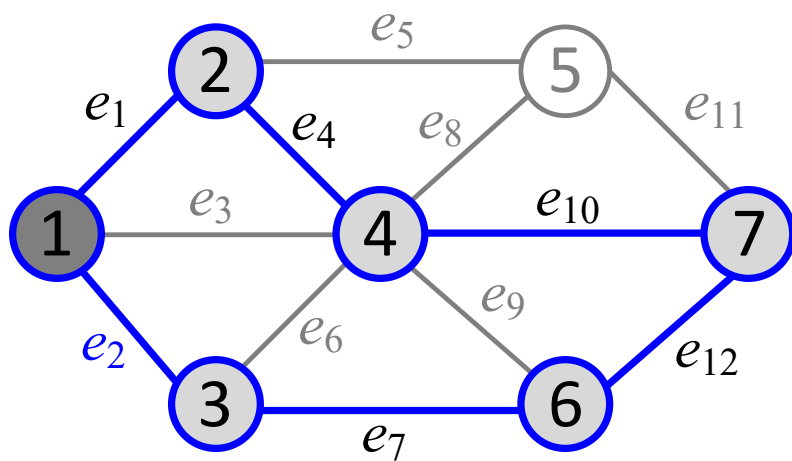


2.5. Estruturas de Percurso

Ciclo

Um ciclo (caminho fechado, *cycle*) em um grafo $G = (V, E)$ é uma sequência $C = v_0 e_1 v_1 \dots v_{k-1} e_k v_0$ cujos termos são alternadamente vértices e arestas de G , sendo que todos os vértices e todas as arestas são distintas, tais que v_{i-1} e v_i são vértices terminais de e_i , $1 \leq i \leq k$. Apenas vértices inicial e final são idênticos ($v_0 = v_k$).

Um ciclo em três ou mais vértices é um grafo simples cujos **vértices** podem ser organizados **em** uma **sequência cíclica**, de modo que dois vértices são adjacentes se forem consecutivos na sequência e não-adjacentes caso contrário.



Ciclo um vértice: um vértice com um **laço**.

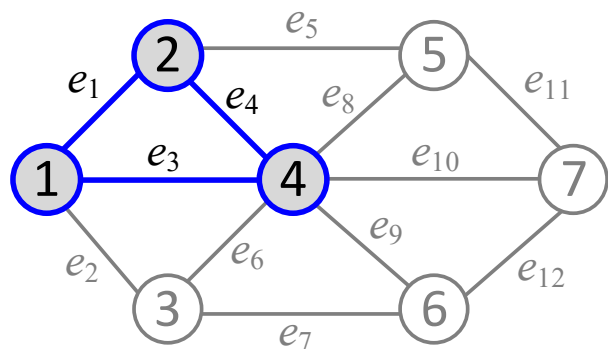
Ciclo dois vértices: dois vértices unidos por um par de **arestas paralelas** (multigrafos) ou par de **arestas disjuntas** entre dois vértices (dígrafos).



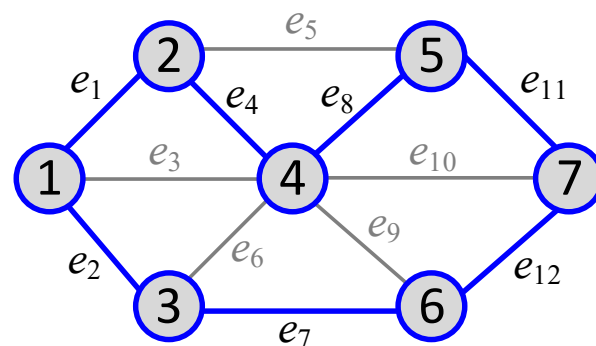
2.5. Estruturas de Percurso

Ciclo

A **cintura** de um grafo corresponde ao comprimento do menor ciclo nele existente.



A **circunferência** de um grafo corresponde ao comprimento do maior ciclo nele existente.

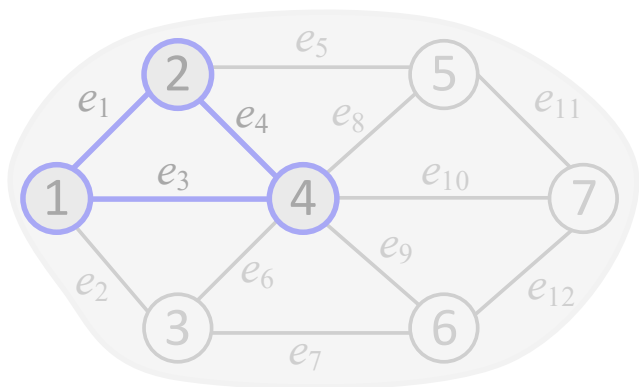




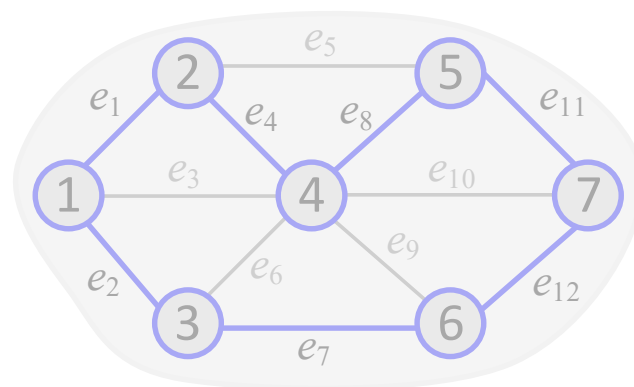
2.5. Estruturas de Percurso

Ciclo

A **cintura** de um grafo corresponde ao comprimento do menor ciclo nele existente.

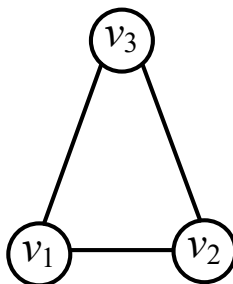


A **circunferência** de um grafo corresponde ao comprimento do maior ciclo nele existente.

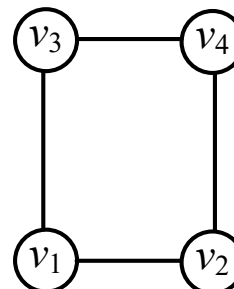


Um **grafo ciclo** de n vértices, denominado C_n , com $n \geq 3$, é um grafo simples que **possui apenas um único ciclo** formado pelos vértices $v_1, v_2, \dots, v_n$, e arestas $v_1v_2, v_2v_3, \dots, v_{n-1}v_n, v_nv_1$.

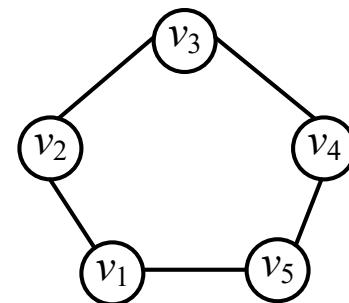
Ex: Grafos ciclos de 3, 4, e 5 vértices.



C_3



C_4



C_5

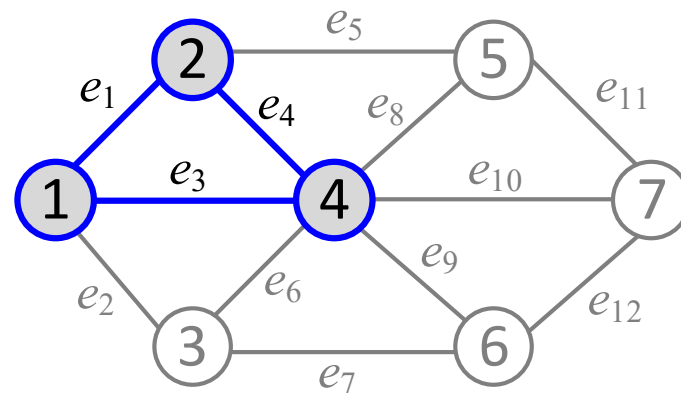


2.5. Estruturas de Percurso

Igualdade de Ciclos

Dois ciclos são considerados iguais se possuem o mesmo conjunto de arestas, mesmo que possuam vértices iniciais diferentes.

Exemplo. O ciclo obtido pela sequência de vértices $v_1v_2v_4v_1$ é o mesmo da sequência $v_2v_4v_1v_2$ e $v_4v_1v_2v_4$.



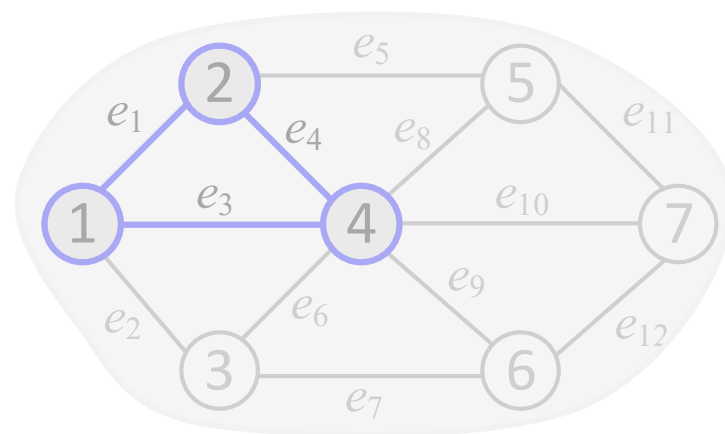


2.5. Estruturas de Percurso

Igualdade de Ciclos

Dois ciclos são considerados iguais se possuem o mesmo conjunto de arestas, mesmo que possuam vértices iniciais diferentes.

Exemplo. O ciclo obtido pela sequência de vértices $v_1v_2v_4v_1$ é o mesmo da sequência $v_2v_4v_1v_2$ e $v_4v_1v_2v_4$.

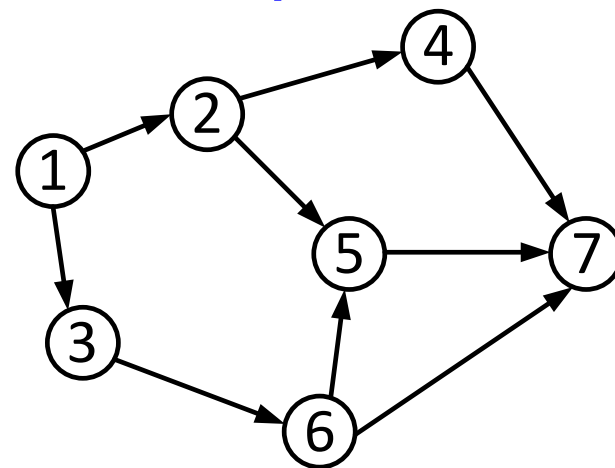


DAG (*Directed Acyclic Graph*)

É um grafo direcionado sem ciclos. A **direção dos arcos não possibilita que se forme um ciclo.**

Toda árvore direcionada é um DAG.

Todo DAG possui ao menos um vértice com grau de entrada zero (fonte).





2.5. Estruturas de Percurso

Terminologia de Percursos

Resumo dos nomes e características dos tipos de subgrafos que constituem percursos em grafos.

Tipo	Repete Arestas?	Repete Vértices?	Término
Passeio (<i>walk</i>)	SIM	SIM	Aberto / Fechado
Trajetó (trilha, <i>trail</i>)	NÃO	SIM	Aberto
Circuito (<i>closed trail</i>)	NÃO	SIM	Fechado
Caminho (<i>path</i>)	NÃO	NÃO	Aberto
Ciclo (<i>closed path</i>)	NÃO	NÃO*	Fechado

* Com exceção dos vértices inicial e final que são idênticos.

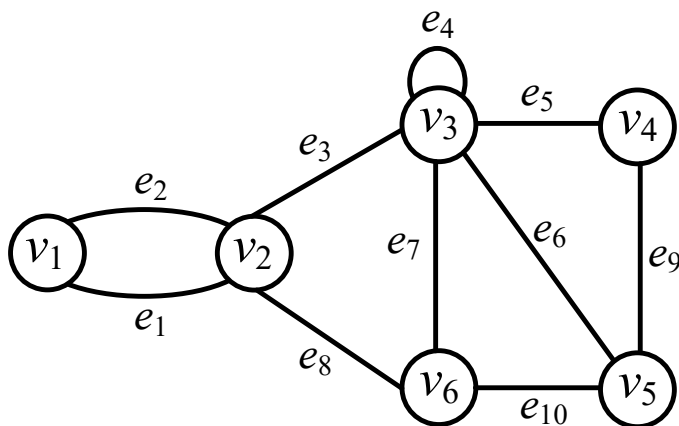


2.5. Estruturas de Percurso

Identificando o Percurso

É possível identificar o tipo de um percurso com base nas suas características.

Exemplo: Identificar o tipo de percurso dado pela sequências de vértices e arestas do grafo abaixo.



$v_1 e_1 v_2 e_3 v_3 e_4 v_3 e_5 v_4$

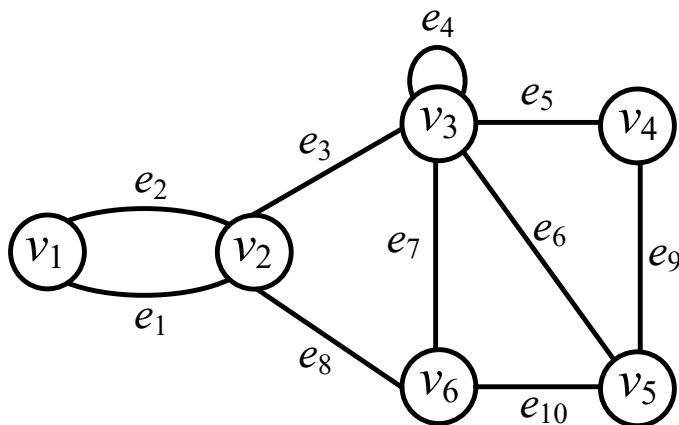


2.5. Estruturas de Percurso

Identificando o Percurso

É possível identificar o tipo de um percurso com base nas suas características.

Exemplo: Identificar o tipo de percurso dado pela sequências de vértices e arestas do grafo abaixo.



$v_1 e_1 v_2 e_3 v_3 e_4 v_3 e_5 v_4$

- Aresta repetida? Não
- Vértice repetido? Sim, v_3
- Fechado? Não

Tipo: **Trajeto**

Tipo	Repete Arestas?	Repete Vértices?	Término
Passeio (walk)	SIM	SIM	Aberto / Fechado
Trajeto (trilha, trail)	NÃO	SIM	Aberto
Circuito (closed trail)	NÃO	SIM	Fechado
Caminho (path)	NÃO	NÃO	Aberto
Ciclo (closed path)	NÃO	NÃO*	Fechado

* Com exceção dos vértices inicial e final que são idênticos.

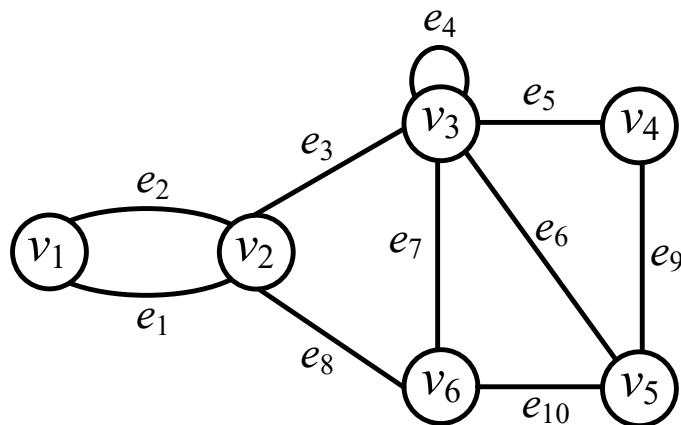


2.5. Estruturas de Percurso

Identificando o Percurso

É possível identificar o tipo de um percurso com base nas suas características.

Exemplo: Identificar o tipo de percurso dado pela sequências de vértices e arestas do grafo abaixo.



$v_1 e_1 v_2 e_3 v_3 e_4 v_3 e_5 v_4$

- Aresta repetida? Não
- Vértice repetido? Sim, v_3
- Fechado? Não

Tipo: Trajeto

Tipo	Repete Arestas?	Repete Vértices?	Término
Passeio (walk)	SIM	SIM	Aberto / Fechado
Trajeto (trilha, trail)	NÃO	SIM	Aberto
Circuito (closed trail)	NÃO	SIM	Fechado
Caminho (path)	NÃO	NÃO	Aberto
Ciclo (closed path)	NÃO	NÃO*	Fechado

* Com exceção dos vértices inicial e final que são idênticos.

$e_1 e_3 e_5 e_5 e_6$

- Aresta repetida? Sim, e_5
- Vértice repetido? Sim, v_3
- Fechado? Não

Tipo: **Passeio Aberto**

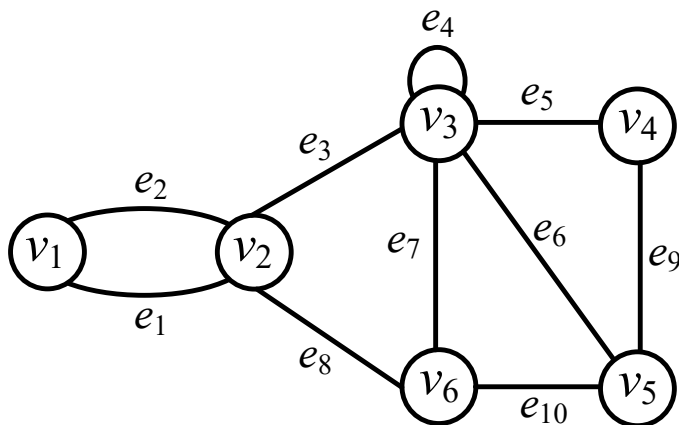


2.5. Estruturas de Percurso

Identificando o Percurso

É possível identificar o tipo de um percurso com base nas suas características.

Exemplo: Identificar o tipo de percurso dado pela sequências de vértices e arestas do grafo abaixo.



$v_2 v_3 v_4 v_5 v_3 v_6 v_2$

- Aresta repetida? Não
- Vértice repetido? Sim, v_2 e v_3
- Fechado? Sim, v_2

Tipo: **Circuito**

Tipo	Repete Arestas?	Repete Vértices?	Término
Passeio (walk)	SIM	SIM	Aberto / Fechado
Trajeto (trilha, trail)	NÃO	SIM	Aberto
Circuito (closed trail)	NÃO	SIM	Fechado
Caminho (path)	NÃO	NÃO	Aberto
Ciclo (closed path)	NÃO	NÃO*	Fechado

* Com exceção dos vértices inicial e final que são idênticos.

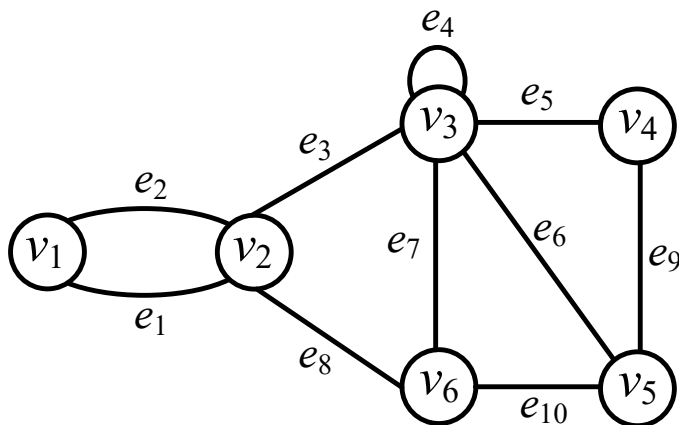


2.5. Estruturas de Percurso

Identificando o Percurso

É possível identificar o tipo de um percurso com base nas suas características.

Exemplo: Identificar o tipo de percurso dado pela sequências de vértices e arestas do grafo abaixo.



$v_2 v_3 v_4 v_5 v_3 v_6 v_2$

- Aresta repetida? Não
- Vértice repetido? Sim, v_2 e v_3
- Fechado? Sim, v_2

Tipo: Circuito

Tipo	Repete Arestas?	Repete Vértices?	Término
Passeio (walk)	SIM	SIM	Aberto / Fechado
Trajeto (trilha, trail)	NÃO	SIM	Aberto
Circuito (closed trail)	NÃO	SIM	Fechado
Caminho (path)	NÃO	NÃO	Aberto
Ciclo (closed path)	NÃO	NÃO*	Fechado

* Com exceção dos vértices inicial e final que são idênticos.

$v_2 v_3 v_4 v_5 v_6 v_2$

- Aresta repetida? Não
- Vértice repetido? Sim, v_2
- Fechado? Sim, v_2

Tipo: Ciclo



2.5. Estruturas de Percurso

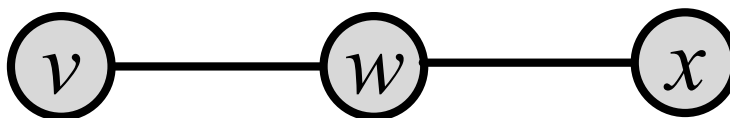
Fecho Transitivo

Na teoria dos grafos o conceito de fecho transitivo pode ser visto como a **identificação** dos **vértices** que atendem a alcançabilidade.

Alcançabilidade auxilia na verificação se é possível chegar à um vértice destino a partir de um vértice origem. É a **capacidade de alcançar** um dado local destino x a partir de um local origem v .

A relação de alcançabilidade é transitiva:

- Se w é alcançável a partir de v
- E se x é alcançável a partir de w
- Então x é alcançável a partir de v

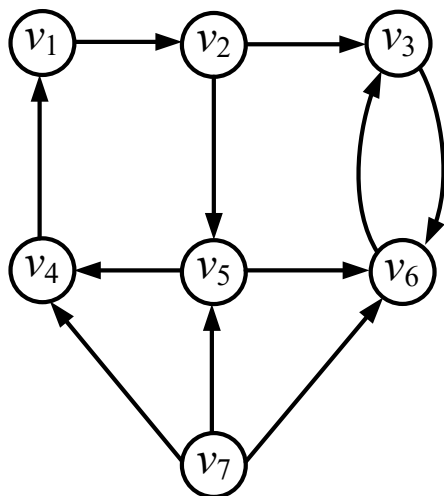




Fecho Transitivo

Fecho Transitivo Direto (FTD)

FTD de um vértice v , denotado $\hat{\Gamma}^+(v)$, é o conjunto de todos os **vértices que podem ser alcançados por algum caminho iniciado em v** . Os vértices em $\hat{\Gamma}^+(v)$ são chamados de descendentes ou **sucessores** de v .



Exemplo. Conjuntos FTD

FTD do vértice v_5 é $\hat{\Gamma}^+(v_5) = \{v_1, v_2, v_3, v_4, v_5, v_6\}$

Obs. O **próprio vértice analisado é incluído** no conjunto FTD, já que ele é alcançável a partir de si mesmo.



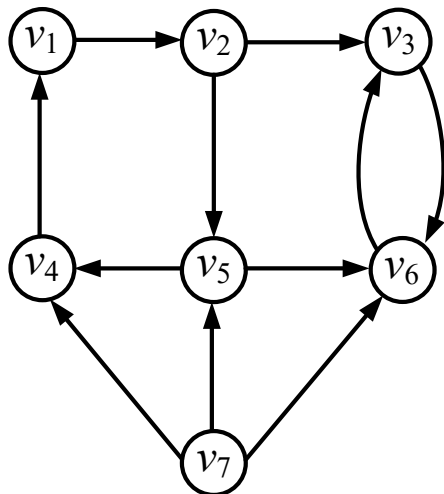
Fecho Transitivo

Fecho Transitivo Direto (FTD)

FTD de um vértice v , denotado $\hat{\Gamma}^+(v)$, é o conjunto de todos os vértices que podem ser alcançados por algum caminho iniciado em v . Os vértices em $\hat{\Gamma}^+(v)$ são chamados de descendentes ou sucessores de v .

Fecho Transitivo Indireto (FTI)

FTI de um vértice v , denotado $\hat{\Gamma}^-(v)$, é o conjunto de todos os **vértices a partir dos quais v é alcançado**. Os vértices em $\hat{\Gamma}^-(v)$ são chamados de ascendentes ou **antecessores** de v .



Exemplo. Conjuntos FTD e FTI

FTD do vértice v_5 é $\hat{\Gamma}^+(v_5) = \{v_1, v_2, v_3, v_4, v_5, v_6\}$

FTI do vértice v_5 é $\hat{\Gamma}^-(v_5) = \{v_1, v_2, v_4, v_5, v_7\}$

Obs. O **próprio vértice analisado é incluído** nos conjuntos FTD e FTI, já que ele é alcançável a partir de si mesmo.



Alcançabilidade – Algoritmo de Warshall

Algoritmo de Warshall para cálculo da matriz de alcançabilidade

warshall (matriz)

```
1  $n$  = quantidade de vertices;  
2  $R$  = cópia de matriz;  
3 for  $k$  de 0 até  $n$  do  
4   for  $i$  de 0 até  $n$  do  
5     for  $j$  de 0 até  $n$  do  
6       if  $R[i][j] == 1 \vee (R[i][k] == 1 \wedge R[k][j] == 1)$  then  
7          $R[i][j] = 1$ ;  
8       end  
9     else  
10       $R[i][j] = R[i][j]$ ;  
11    end  
12  end  
13 end  
14 end
```

Tem como entrada uma matriz de adjacências.

Verifica todos os caminhos com k vértices intermediários entre cada par de vértices i e j .



Alcançabilidade – Algoritmo de Warshall

Algoritmo de Warshall para cálculo da matriz de alcançabilidade

warshall (matriz)

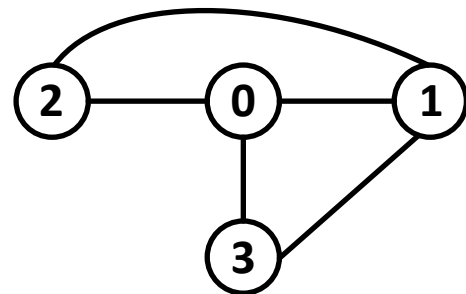
```
1  n = quantidade de vertices;  
2  R = cópia de matriz;  
3  for k de 0 até n do  
4      for i de 0 até n do  
5          for j de 0 até n do  
6              if R[i][j] == 1 ∨ (R[i][k] == 1 ∧ R[k][j] == 1) then  
7                  R[i][j] = 1;  
8              end  
9              else  
10                 R[i][j] = R[i][j];  
11             end  
12         end  
13     end  
14 end
```

Em um grafo simples e conexo **cada vértice é alcançável a partir de qualquer outro** vértice.

Cada iteração responde: “Se eu puder usar o vértice k , consigo criar novos caminhos?”. Isso é feito para todos os pares (i, j) .

Obtém uma **matriz de alcançabilidade** (ou matriz de caminhos) a partir de relações par-a-par de matrizes de adjacência.

Grafo Simples



$$R(G) = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$



Alcançabilidade – Algoritmo de Warshall

Algoritmo de Warshall para cálculo da matriz de alcançabilidade

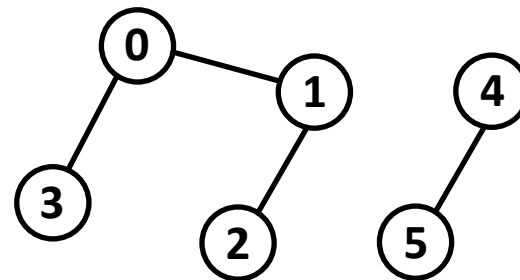
warshall (matriz)

```
1  n = quantidade de vertices;  
2  R = cópia de matriz;  
3  for k de 0 até n do  
4      for i de 0 até n do  
5          for j de 0 até n do  
6              if R[i][j] == 1 ∨ (R[i][k] == 1 ∧ R[k][j] == 1) then  
7                  R[i][j] = 1;  
8              end  
9              else  
10                 R[i][j] = R[i][j];  
11             end  
12         end  
13     end  
14 end
```

Possibilita identificar a alcançabilidade entre elementos de componentes conexos.

Obtém uma matriz de alcançabilidade (ou matriz de caminhos) a partir de relações par-a-par de matrizes de adjacência.

Grafo Desconexo



$$R(G) = \begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 \end{bmatrix}$$

→ Elementos do componente conexo {0, 1, 2, 3}

→ Elementos do componente conexo {4, 5}



2.5. Estruturas de Percurso

Alcançabilidade – Algoritmo de Warshall

Algoritmo de Warshall para cálculo da matriz de alcançabilidade

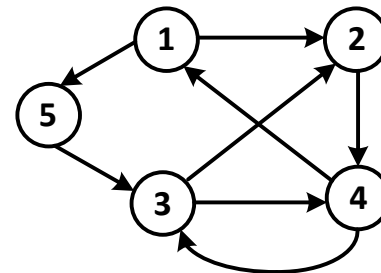
warshall (matriz)

```
1  n = quantidade de vertices;
2  R = cópia de matriz;
3  for k de 0 até n do
4      for i de 0 até n do
5          for j de 0 até n do
6              if R[i][j] == 1 ∨ (R[i][k] == 1 ∧ R[k][j] == 1) then
7                  R[i][j] = 1;
8              end
9              else
10                 R[i][j] = R[i][j];
11             end
12         end
13     end
14 end
```

Um dígrafo é fortemente conexo (todos os vértices alcançam os demais) se sua matriz de alcançabilidade tem todos os elementos iguais a 1.

Obtém uma matriz de alcançabilidade (ou matriz de caminhos) a partir de relações par-a-par de matrizes de adjacência.

Dígrafo f-conexo



$$R(G) = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$



2.5. Estruturas de Percurso

Alcançabilidade – Algoritmo de Warshall

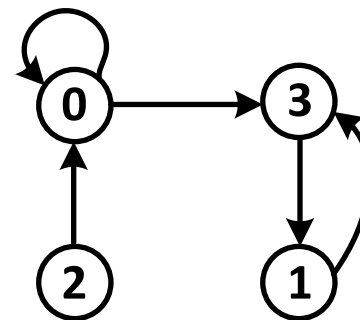
Algoritmo de Warshall para cálculo da matriz de alcançabilidade

warshall (matriz)

```

1  n = quantidade de vertices;
2  R = cópia de matriz;
3  for k de 0 até n do
4      for i de 0 até n do
5          for j de 0 até n do
6              if R[i][j] == 1 ∨ (R[i][k] == 1 ∧ R[k][j] == 1) then
7                  R[i][j] = 1;
8              end
9              else
10                 R[i][j] = R[i][j];
11             end
12         end
13     end
14 end
  
```

Ex. Digrafo



$$R_0 = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

$$R_1 = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & & & \\ 1 & & & \\ 0 & & & \end{bmatrix}$$

Diagram illustrating the update of R_1 from R_0 . Red dashed arrows show the propagation of reachability: from $R_0[0][3] = 1$ to $R_1[0][1] = 1$ (via $R_0[3][1] = 1$), and from $R_0[2][0] = 1$ to $R_1[2][1] = 1$ (via $R_0[0][1] = 1$). The cell $R_1[1][1]$ is highlighted with a red circle.



2.5. Estruturas de Percurso

Alcançabilidade – Algoritmo de Warshall

Algoritmo de Warshall para cálculo da matriz de alcançabilidade

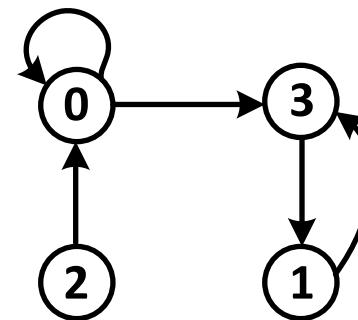
warshall (matriz)

```

1  n = quantidade de vertices;
2  R = cópia de matriz;
3  for k de 0 até n do
4      for i de 0 até n do
5          for j de 0 até n do
6              if R[i][j] == 1 ∨ (R[i][k] == 1 ∧ R[k][j] == 1) then
7                  R[i][j] = 1;
8              end
9              else
10                 R[i][j] = R[i][j];
11             end
12         end
13     end
14 end

```

Ex. Digrafo



$$R_0 = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

$$R_1 = \begin{bmatrix} \mathbf{1} & \mathbf{0} & \mathbf{0} & \mathbf{1} \\ \mathbf{0} & 0 & 0 & 1 \\ \mathbf{1} & 0 & 0 & \mathbf{1} \\ \mathbf{0} & 1 & 0 & 0 \end{bmatrix}$$



2.5. Estruturas de Percurso

Alcançabilidade – Algoritmo de Warshall

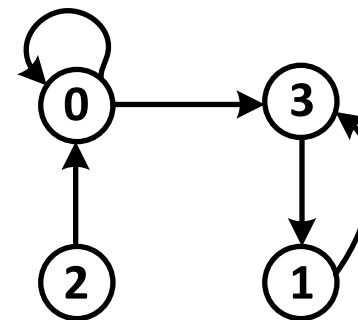
Algoritmo de Warshall para cálculo da matriz de alcançabilidade

warshall (matriz)

```

1  n = quantidade de vertices;
2  R = cópia de matriz;
3  for k de 0 até n do
4      for i de 0 até n do
5          for j de 0 até n do
6              if R[i][j] == 1 ∨ (R[i][k] == 1 ∧ R[k][j] == 1) then
7                  R[i][j] = 1;
8              end
9              else
10                 R[i][j] = R[i][j];
11             end
12         end
13     end
14 end
  
```

Ex. Digrafo



$$R_0 = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

$$R_1 = \begin{bmatrix} \mathbf{1} & \mathbf{0} & \mathbf{0} & \mathbf{1} \\ \mathbf{0} & 0 & 0 & 1 \\ \mathbf{1} & 0 & 0 & \mathbf{1} \\ \mathbf{0} & 1 & 0 & 0 \end{bmatrix}$$

$$R_2 = \begin{bmatrix} 1 & \mathbf{0} & 0 & 1 \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{1} \\ 1 & \mathbf{0} & 0 & 1 \\ 0 & \mathbf{1} & 0 & \mathbf{1} \end{bmatrix}$$

$$R_3 = \begin{bmatrix} 1 & 0 & \mathbf{0} & 1 \\ 0 & 0 & \mathbf{0} & 1 \\ \mathbf{1} & \mathbf{0} & \mathbf{0} & \mathbf{1} \\ 0 & 1 & \mathbf{0} & 1 \end{bmatrix}$$

$$R_4 = \begin{bmatrix} 1 & \mathbf{1} & 0 & \mathbf{1} \\ 0 & \mathbf{1} & 0 & \mathbf{1} \\ 1 & \mathbf{1} & 0 & \mathbf{1} \\ \mathbf{0} & \mathbf{1} & 0 & \mathbf{1} \end{bmatrix}$$



Alcançabilidade – Algoritmo de Warshall

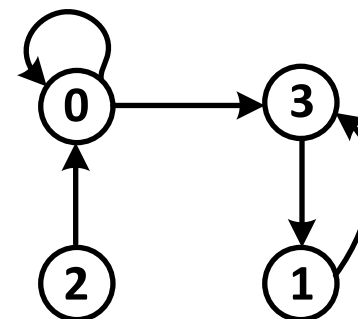
Algoritmo de Warshall para cálculo da matriz de alcançabilidade

warshall (matriz)

```

1  n = quantidade de vertices;
2  R = cópia de matriz;
3  for k de 0 até n do
4      for i de 0 até n do
5          for j de 0 até n do
6              if R[i][j] == 1 ∨ (R[i][k] == 1 ∧ R[k][j] == 1) then
7                  R[i][j] = 1;
8              end
9              else
10                 R[i][j] = R[i][j];
11             end
12         end
13     end
14 end
    
```

Ex. Digrafo



$$R_0 = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

$$R_1 = \begin{bmatrix} \mathbf{1} & \mathbf{0} & \mathbf{0} & \mathbf{1} \\ \mathbf{0} & 0 & 0 & 1 \\ \mathbf{1} & 0 & 0 & \mathbf{1} \\ \mathbf{0} & 1 & 0 & 0 \end{bmatrix}$$

$$R_2 = \begin{bmatrix} 1 & \mathbf{0} & 0 & 1 \\ \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{1} \\ 1 & \mathbf{0} & 0 & 1 \\ 0 & \mathbf{1} & 0 & \mathbf{1} \end{bmatrix}$$

$$R_3 = \begin{bmatrix} 1 & 0 & \mathbf{0} & 1 \\ 0 & 0 & \mathbf{0} & 1 \\ \mathbf{1} & \mathbf{0} & \mathbf{0} & \mathbf{1} \\ 0 & 1 & \mathbf{0} & 1 \end{bmatrix}$$

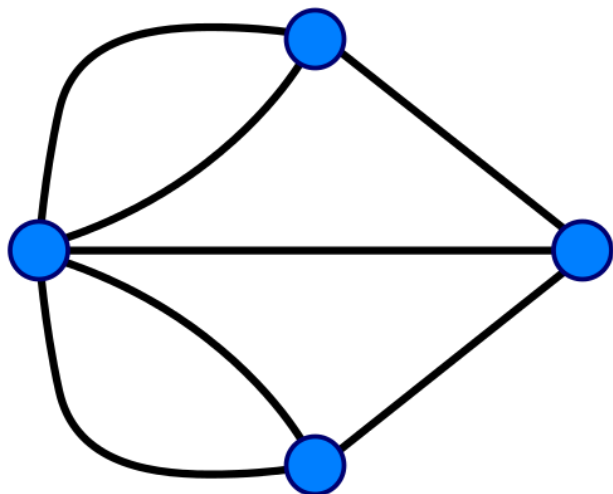
$$R_4 = \begin{bmatrix} 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 \end{bmatrix}$$



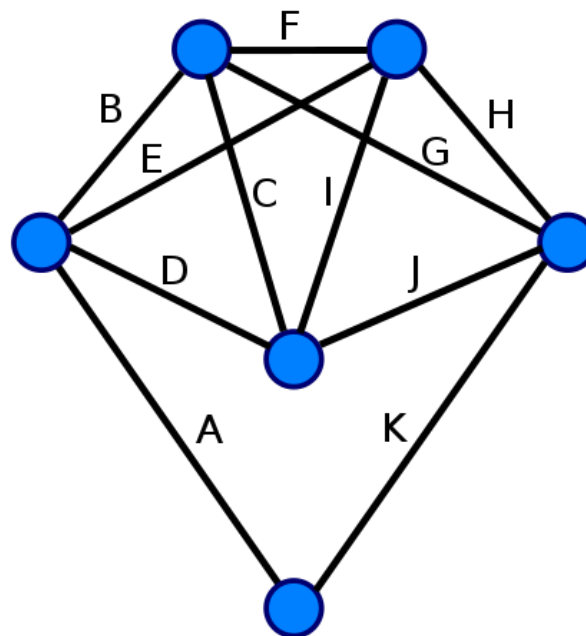
2.5. Estruturas de Percurso

Circuito Euleriano

Caminho Euleriano é um caminho em um grafo que **visita toda aresta exatamente uma vez**. **Circuito Euleriano** em um grafo G é dado pela sequência de vértices e arestas adjacentes que **inicia e termina no mesmo vértice**, passando **pelo menos uma vez por cada vértice** e exatamente **uma única vez por cada aresta** de G .



Grafo não-Euleriano



Grafo Euleriano



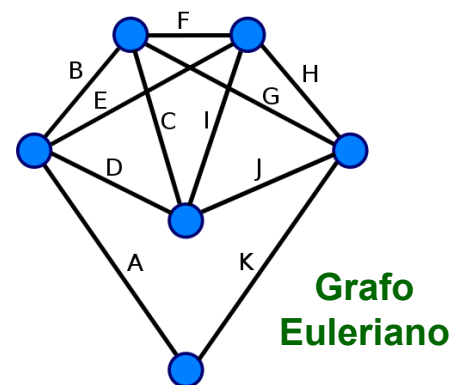
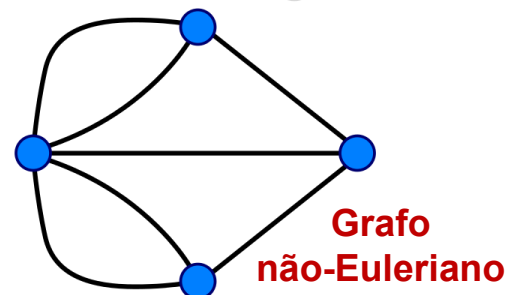
2.5. Estruturas de Percurso

Circuito Euleriano

Caminho Euleriano é um caminho em um grafo que visita toda aresta exatamente uma vez. **Circuito Euleriano** em um grafo G é dado pela sequência de vértices e arestas adjacentes que inicia e termina no mesmo vértice, passando pelo menos uma vez por cada vértice e exatamente uma única vez por cada aresta de G .

Critérios para que um grafo G seja Euleriano:

- G é um grafo conexo;
- Teorema de Euler: G é euleriano sse todos os seus vértices possuam grau par;
- G é não-euleriano sse existem **um, três ou mais vértices de grau ímpar**;
- G é semi-euleriano sse existem exatamente dois vértices de grau ímpar.





2.5. Estruturas de Percurso

Circuito Euleriano

Caminho Euleriano é um caminho em um grafo que visita toda aresta exatamente uma vez. **Circuito Euleriano** em um grafo G é dado pela sequência de vértices e arestas adjacentes que inicia e termina no mesmo vértice, passando pelo menos uma vez por cada vértice e exatamente uma única vez por cada aresta de G .

Crítérios para que um grafo G seja Euleriano:

- G é um grafo conexo;
- Teorema de Euler: G é euleriano sse todos os seus vértices possuírem grau par;
- G é não-euleriano sse existem um, três ou mais vértices de grau ímpar;
- G é semi-euleriano sse existem exatamente dois vértices de grau ímpar.

Algoritmo para verificar se o grafo possui um caminho Euleriano

caminhoEuleriano(G)

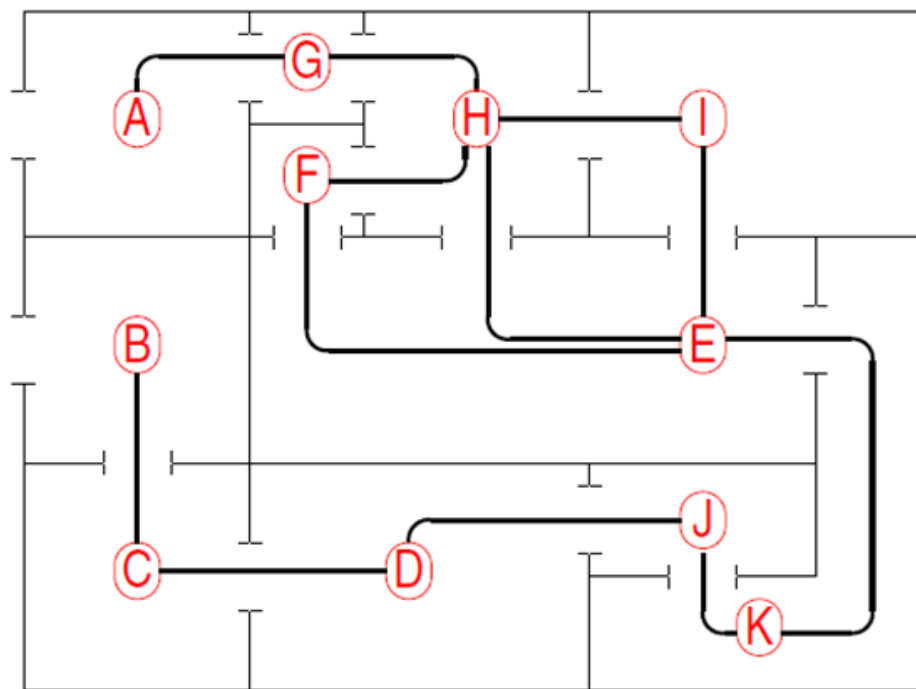
```
1   $n$  = quantidade de vertices;
2   $total$  = 0;
3   $i$  = 0;
4  while ( $total \leq 2$ )  $\wedge$  ( $i < n$ ) do
5    | obtém grau do vértice  $i$ ;
6    | if grau é ímpar then
7    |   |  $total = total + 1$ ;
8    |   | end
9    |   |  $i = i + 1$ ;
10 end
11 if  $total > 2$  then
12 |    $False$ ;
13 end
14 else
15 |    $True$ ;
16 end
```



2.5. Estruturas de Percurso

Circuito Euleriano

Exemplo: Uma casa possui uma divisão representada pela planta abaixo. É possível um robô de limpeza **sair do cômodo A, terminar no cômodo B e passar por todas as portas da casa exatamente uma única vez?**

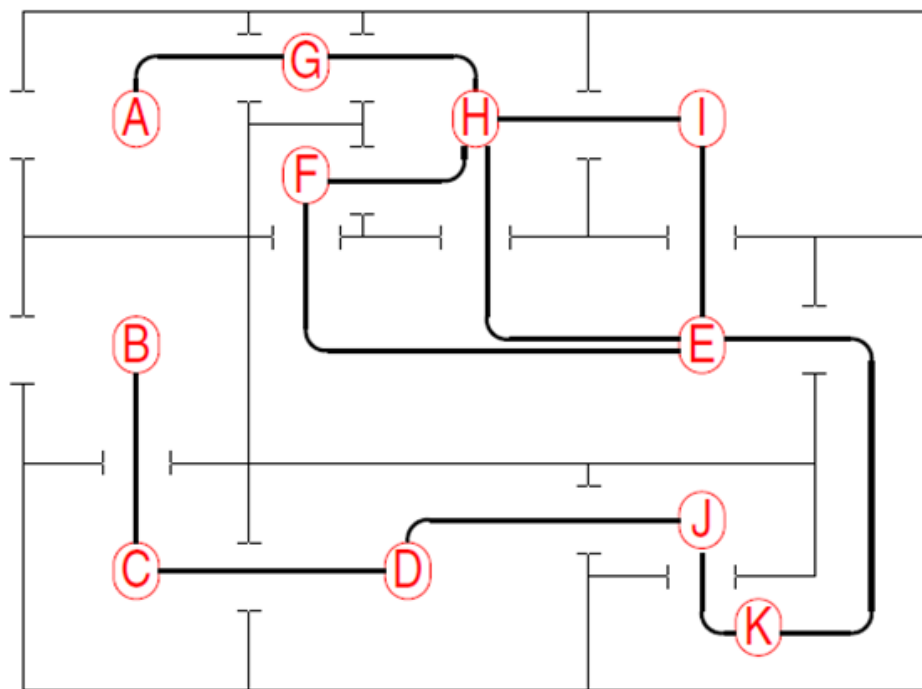




2.5. Estruturas de Percurso

Circuito Euleriano

Exemplo: Uma casa possui uma divisão representada pela planta abaixo. É possível um robô de limpeza **sair do cômodo A, terminar no cômodo B e passar por todas as portas da casa exatamente uma única vez?**



Cada vértice do grafo tem um grau par (OK), exceto os vértices A e B que têm grau 1 (OK, dois vértices apenas).

Existe um **caminho Euleriano** de A para B dado pela sequência:

A G H F E I H E K J D C B

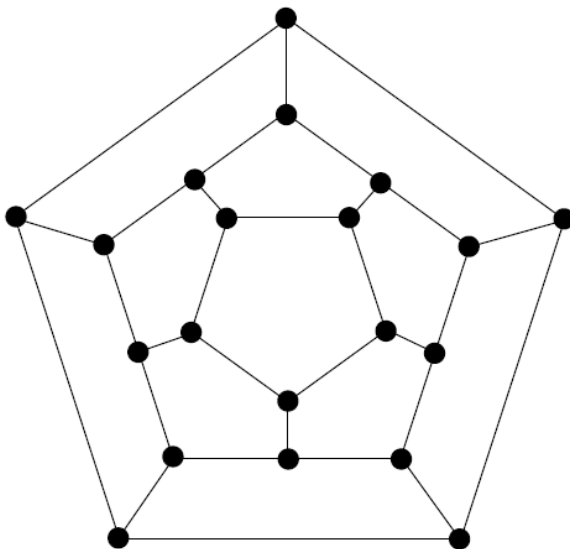


2.5. Estruturas de Percurso

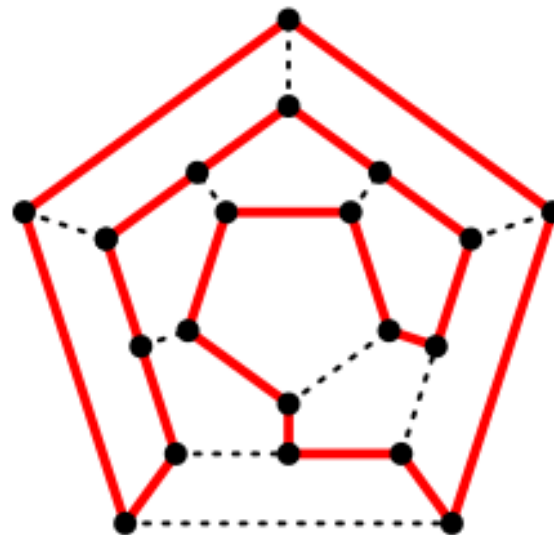
Ciclo Hamiltoniano

Caminho Hamiltoniano é um caminho que passa exatamente uma vez por cada vértice de um grafo. **Ciclo Hamiltoniano** é um caminho hamiltoniano que retorna ao vértice inicial.

Um **grafo hamiltoniano** é aquele que possui um ciclo hamiltoniano. Um grafo **Semi-Hamiltoniano** é aquele que possui um caminho hamiltoniano.



Grafo Hamiltoniano



Ciclo Hamiltoniano

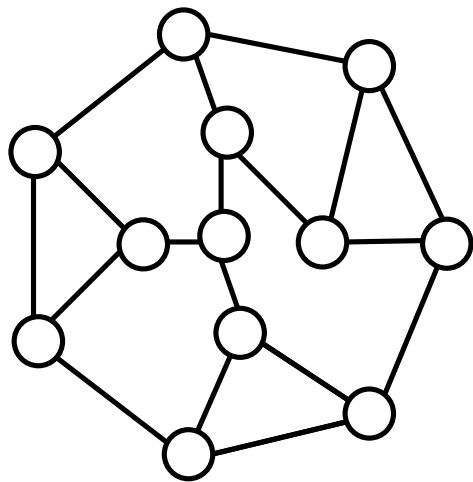


2.5. Estruturas de Percurso

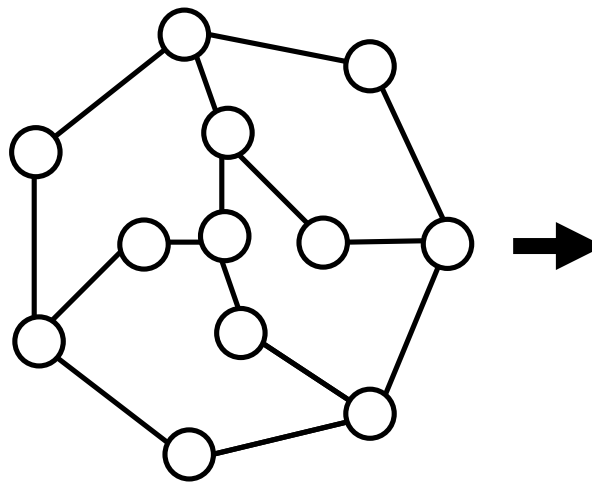
Ciclo Hamiltoniano

Caminho Hamiltoniano é um caminho que passa exatamente uma vez por cada vértice de um grafo. **Ciclo Hamiltoniano** é um caminho hamiltoniano que retorna ao vértice inicial.

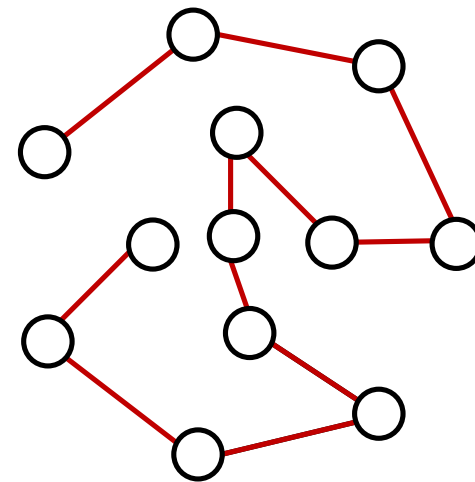
Um **grafo hamiltoniano** é aquele que possui um ciclo hamiltoniano. Um grafo **Semi-Hamiltoniano** é aquele que possui um caminho hamiltoniano.



grafo
Hamiltoniano



grafo
semi-Hamiltoniano



caminho
Hamiltoniano



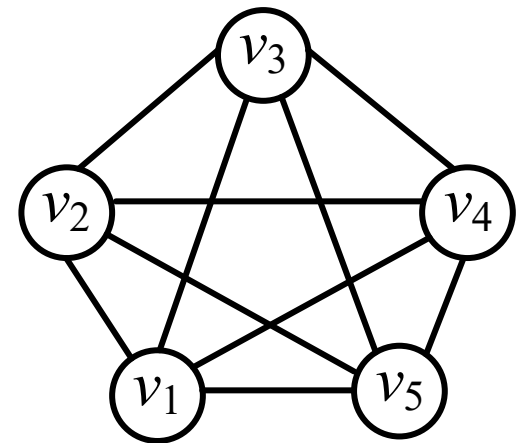
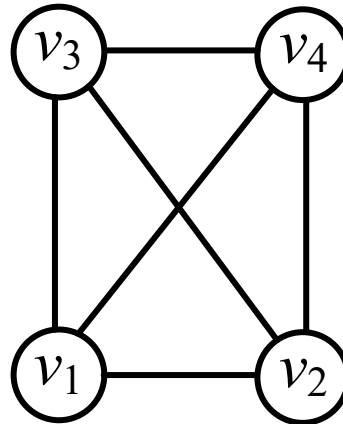
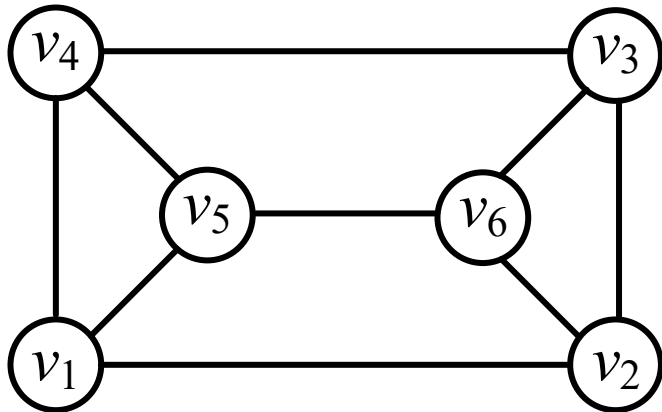
2.5. Estruturas de Percurso

Ciclo Hamiltoniano

Não existe um teorema trivial que indique se um grafo é Hamiltoniano, nem se conhece um algoritmo eficiente (polinomial) para achar um ciclo Hamiltoniano. Alguns teoremas foram sugeridos:

Teorema de Dirac (1952)

Um grafo $G = (V, A)$ com $|V| \geq 3$ e $d(v) \geq |V| / 2$ para todo $v \in V$ é hamiltoniano, onde $d(v)$ corresponde ao grau do vértice.





2.5. Estruturas de Percurso

Ciclo Hamiltoniano

Não existe um teorema trivial que indique se um grafo é Hamiltoniano, nem se conhece um algoritmo eficiente (polinomial) para achar um ciclo Hamiltoniano. Alguns teoremas foram sugeridos:

Teorema de Ore (1960)

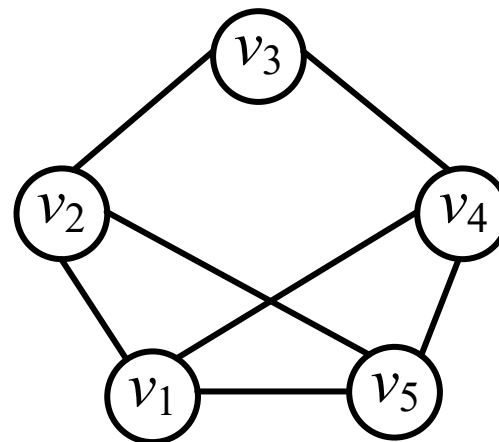
Em um grafo hamiltoniano a soma dos graus de cada par de vértices não adjacentes é no mínimo $|V|$, ou seja, $d(u) + d(v) \geq |V|$ para todo $u, v \in V$ não adjacente.

$|V| = 5$ vértices

$$d(v_1) + d(v_3) = 3 + 2 = 5 \text{ (OK)}$$

$$d(v_2) + d(v_4) = 3 + 3 = 6 \text{ (OK)}$$

$$d(v_3) + d(v_5) = 2 + 3 = 5 \text{ (OK)}$$





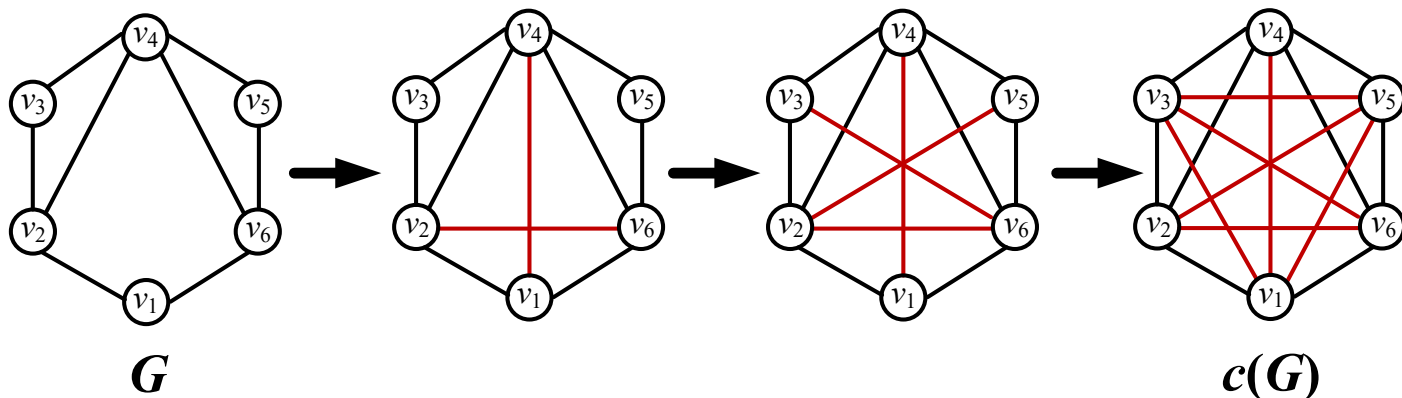
2.5. Estruturas de Percurso

Ciclo Hamiltoniano

Não existe um teorema trivial que indique se um grafo é Hamiltoniano, nem se conhece um algoritmo eficiente (polinomial) para achar um ciclo Hamiltoniano. Alguns teoremas foram sugeridos:

Teorema de Bondy & Chvátal (1976)

Um grafo G é Hamiltoniano se seu Fecho Hamiltoniano, denominado $c(G)$, for um grafo completo. Para obter $c(G)$, arestas são sucessivamente adicionadas entre pares de vértice u e v não adjacentes cuja soma dos graus seja pelo menos $|V|$, até que os pares não-adjacentes não mais existam.



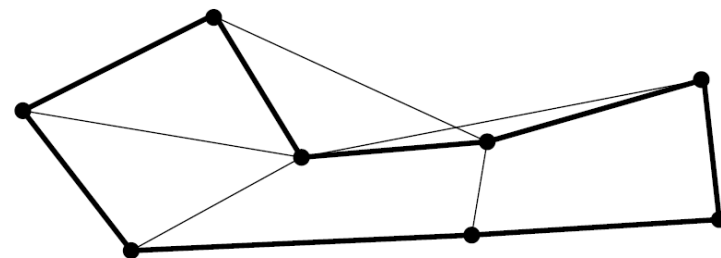


2.5. Estruturas de Percurso

Ciclo Hamiltoniano

Determinando se um grafo não possui um ciclo Hamiltoniano: Se um grafo G tem um ciclo Hamiltoniano então G tem um subgrafo H com as propriedades:

1. H contém cada vértice de G
2. H é conexo
3. H tem o mesmo número de arestas e de vértices
4. Cada vértice de H tem grau 2.



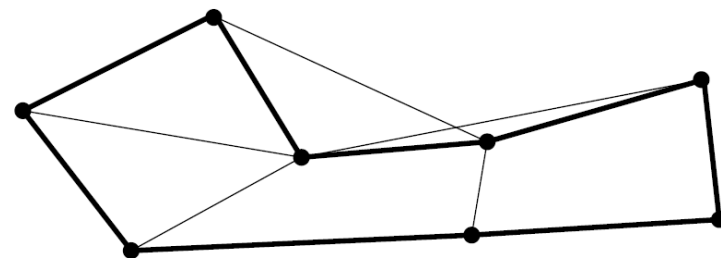


2.5. Estruturas de Percurso

Ciclo Hamiltoniano

Determinando se um grafo não possui um ciclo Hamiltoniano: Se um grafo G tem um ciclo Hamiltoniano então G tem um subgrafo H com as propriedades:

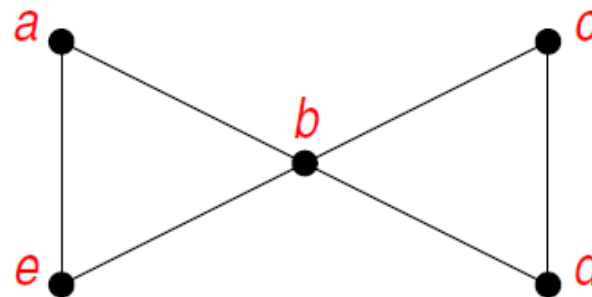
1. H contém cada vértice de G
2. H é conexo
3. H tem o mesmo número de arestas e de vértices
4. Cada vértice de H tem grau 2.



Por contraposição se um grafo G não tem um subgrafo H com as propriedades (1) a (4) então G não possui um circuito Hamiltoniano.

Exemplo:

- $d(b) = 4$ e cada vértice de H tem grau 2
- Remover duas arestas de b para criar H
- Os outros vértices terão grau menor que 2 se qualquer aresta for removida de b .

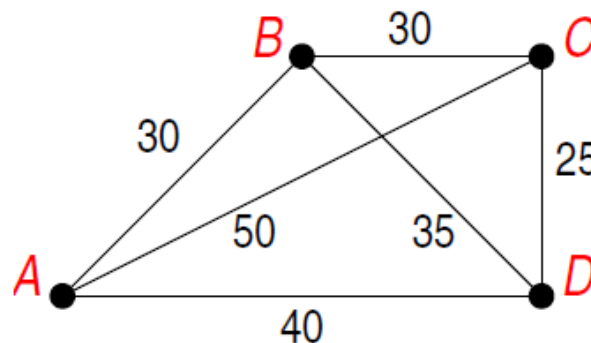




2.5. Estruturas de Percurso

Ciclo Hamiltoniano

Exemplo: Um caixeiro viajante precisa visitar cada cidade exatamente uma única vez e voltar a cidade inicial. Que rota deve ser escolhida para minimizar o total da distância percorrida?





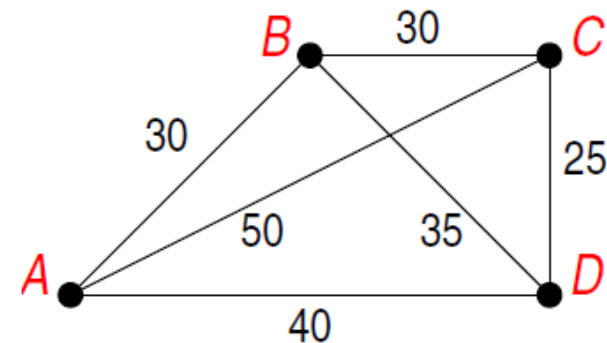
2.5. Estruturas de Percurso

Ciclo Hamiltoniano

Exemplo: Um caixeiro viajante precisa visitar cada cidade exatamente uma única vez e voltar a cidade inicial. **Que rota deve ser escolhida para minimizar o total da distância percorrida?**

Possível solução:

- Enumerar todos os possíveis ciclos Hamiltonianos com início e término em A;
- Calcular a distância de cada ciclo e selecionar o menor – rotas ABCDA ou ADCBA têm a menor distância total.



Rota	Distância (km)
ABCDA	$30 + 30 + 25 + 40 = 125$
ABDCA	$30 + 35 + 25 + 50 = 140$
ACBDA	$50 + 30 + 35 + 40 = 155$
ACDBA	$50 + 25 + 35 + 30 = 140$
ADBCA	$40 + 35 + 30 + 50 = 155$
ADCBA	$40 + 25 + 30 + 30 = 125$

Contudo, esta **solução não é escalável**.
Para o grafo K_{30} existem $29! = 8,84 \times 10^{30}$ ciclos Hamiltonianos diferentes.



2.6. Conectividade



2.6. Conectividade

Definição

Conectividade está relacionada a um número mínimo de elementos (vértices ou arestas) que precisam ser removidos para desconectar os vértices restantes uns dos outros. A conectividade de um grafo é uma medida da robustez de uma rede.

Formalmente: Dois vértices v_i e v_j de um grafo G estão **conectados** sse **existe um caminho de v_i para v_j** . Um grafo G é conexo sse dado um par qualquer de vértice v_i e v_j em G , existe um caminho de v_i para v_j .

G é conexo $\Leftrightarrow \forall$ vértices $v_i, v_j \in V(G), \exists$ um caminho de v_i para v_j .



2.6. Conectividade

Definição

Conectividade está relacionada a um número mínimo de elementos (vértices ou arestas) que precisam ser removidos para desconectar os vértices restantes uns dos outros. A conectividade de um grafo é uma medida da robustez de uma rede.

Formalmente: Dois vértices v_i e v_j de um grafo G estão **conectados** sse existe um caminho de v_i para v_j . Um grafo G é **conexo** sse dado um par qualquer de vértice v_i e v_j em G , existe um caminho de v_i para v_j .

G é conexo $\Leftrightarrow \forall$ vértices $v_i, v_j \in V(G), \exists$ um caminho de v_i para v_j .

Lemas da Conectividade. Seja um grafo G :

- a) Se G é conexo, então quaisquer dois vértices distintos de G podem ser conectados por um caminho (*path*).
- b) Se vértices v_i e v_j são parte de um circuito de G e uma aresta é removida do circuito, ainda assim existe um caminho v_i para v_j em G .
- c) Se G é conexo e contém um circuito, então uma aresta do circuito pode ser removida sem desconectar G .



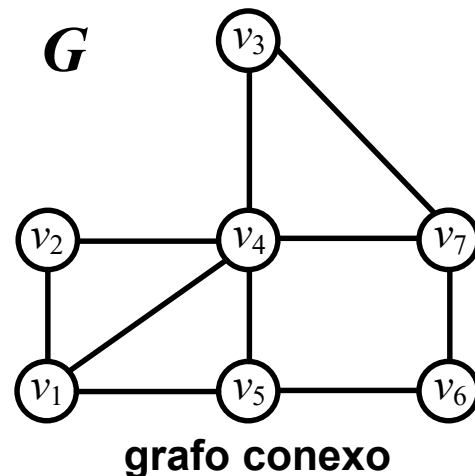
2.6. Conectividade

Definição

Grafo Não Direcionado (GND)

Em um GND **conexo**, todos os vértices são **alcançáveis** a partir de qualquer outro (vale para grafos simples, multigrafos, pseudografos).

Sempre é possível fazer um passeio fechado que inclua todos os vértices.





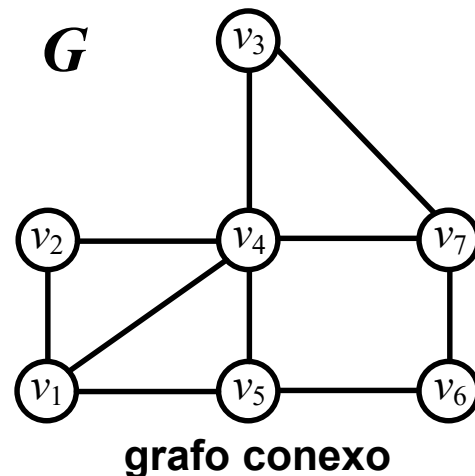
2.6. Conectividade

Definição

Grafo Não Direcionado (GND)

Em um GND conexo, todos os vértices são alcançáveis a partir de qualquer outro (vale para grafos simples, multigrafos, pseudografos).

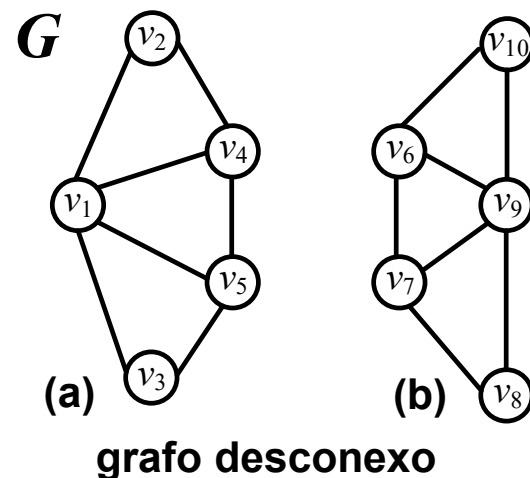
Sempre é possível fazer um passeio fechado que inclua todos os vértices.



Grafo Desconexo

É aquele no qual não existe um caminho em pelo menos um par de seus vértices.

O grafo possui **duas componentes não existindo uma ligação entre elas**. Um vértice da componente (a) não alcança um vértice da componente (b).

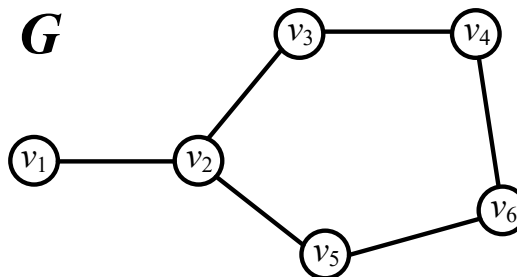
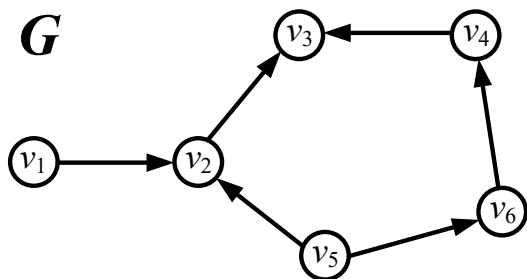




Conectividade em Grafos Direcionados

Grafo simplesmente conexo (s-conexo)

Se G é um grafo direcionado então ele é considerado conexo quando o seu **grafo não-direcionado subjacente é conexo**. O grafo subjacente é o resultante quando a orientação dos arcos de G é ignorada.



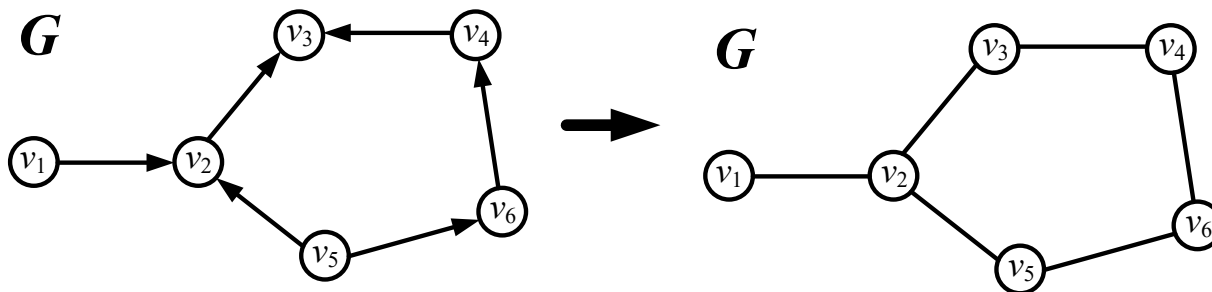
grafo não-orientado subjacente
(indica se o grafo é conexo
desconsiderando as direções)



Conectividade em Grafos Direcionados

Grafo simplesmente conexo (s-conexo)

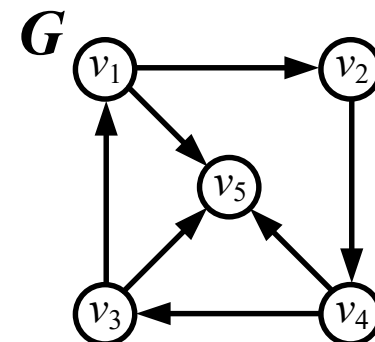
Se G é um grafo direcionado então ele é considerado conexo quando o seu grafo não-direcionado subjacente é conexo. O grafo subjacente é o resultante quando a orientação dos arcos de G é ignorada.



grafo não-orientado subjacente
(indica se o grafo é conexo
desconsiderando as direções)

Grafo Semi-Fortemente Conexo (sf-conexo)

Se G é um grafo direcionado então ele é considerado sf-conexo se para cada par de vértices (v_i, v_j) , existe um **caminho** de v_i para v_j **ou** de v_j para v_i (**assimétrico**).



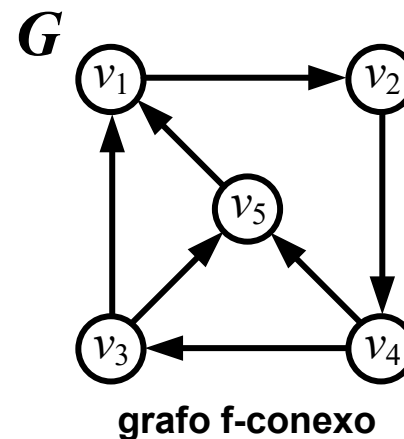
grafo sf-conexo



Conectividade em Grafos Direcionados

Grafo Fortemente Conexo (f-conexo)

Se G é um grafo direcionado então ele é considerado f-conexo se para cada par de vértices (v_i, v_j) , existe um **caminho** de v_i para v_j e de v_j para v_i (**simétrico**).

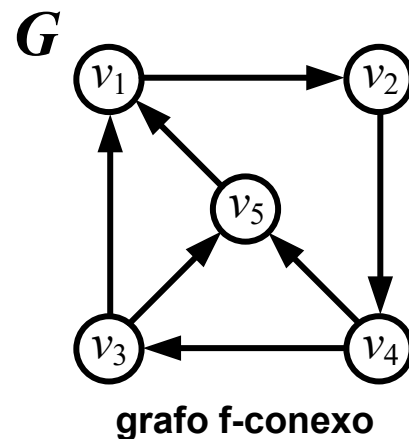




Conectividade em Grafos Direcionados

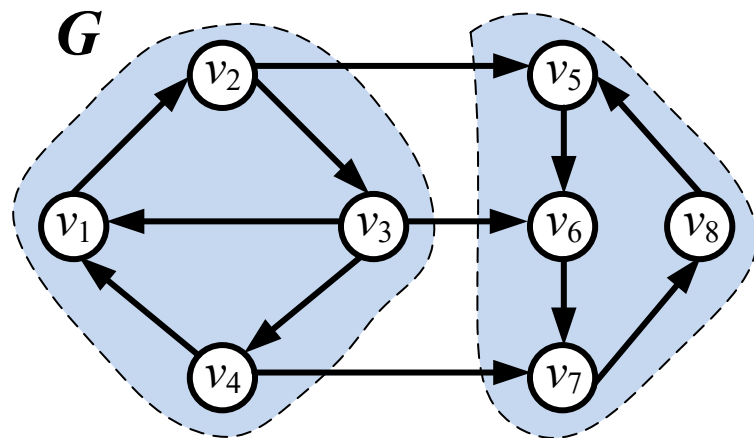
Grafo Fortemente Conexo (f-conexo)

Se G é um grafo direcionado então ele é considerado f-conexo se para cada par de vértices (v_i, v_j) , existe um caminho de v_i para v_j e de v_j para v_i (simétrico).



Componentes Fortemente Conexos

Se G é um grafo direcionado, componentes fortemente conexos são subgrafos maximais f-conexos (todos os vértices de um mesmo componente são alcançáveis entre si).

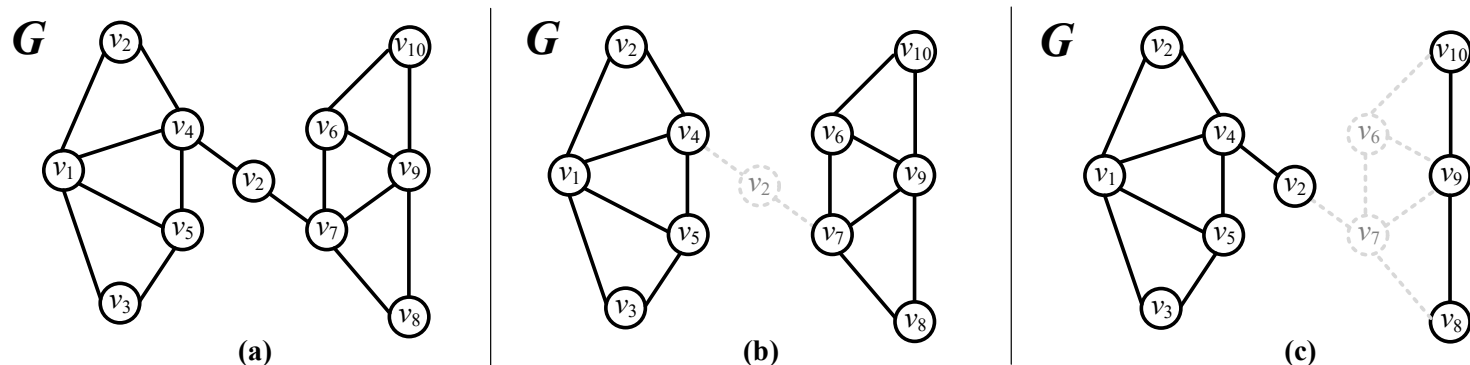




2.6. Conectividade

Conectividade em Vértices

Aplicado a **grafos não-direcionados**. A conectividade em vértices $\kappa(G)$ de um grafo G corresponde a **menor quantidade de vértices cuja remoção torna G desconexo**.

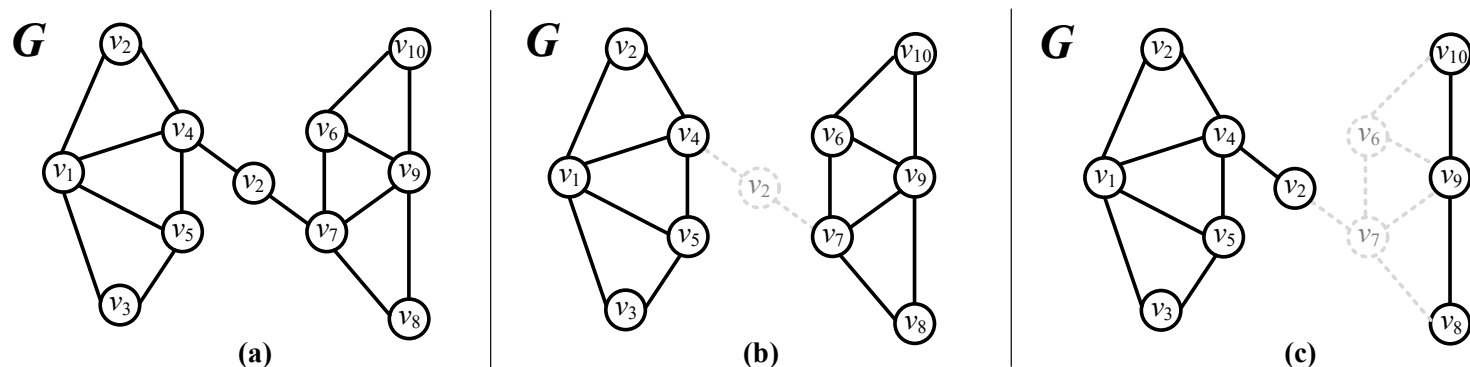


Um grafo G (a), em (b) G após a remoção de um vértice (**vértice de corte**) e em (c) com a remoção de dois vértices (**conjunto separador**). Neste caso, a conectividade de G é $\kappa(G) = 1$.



Conectividade em Vértices

Aplicado a grafos não-direcionados. A conectividade em vértices $\kappa(G)$ de um grafo G corresponde a menor quantidade de vértices cuja remoção torna G desconexo.



Um grafo G (a), em (b) G após a remoção de um vértice (vértice de corte) e em (c) com a remoção de dois vértices (conjunto separador). Neste caso, a conectividade de G é $\kappa(G) = 1$.

- Nos grafos completos com $|V|$ vértices, $\kappa(G) = |V| - 1$
- Nos grafos não-completos existirá pelo menos um par (v_i, v_j) de vértices não-adjacentes, ocasionando em uma $\kappa(G) = |V| - 2 \forall G \neq K_n$
- O limite superior para $\kappa(G)$ em qualquer grafo é $\kappa(G) \leq \delta(G)$, onde δ corresponde ao menor grau em um GND (grafo não direcionado).

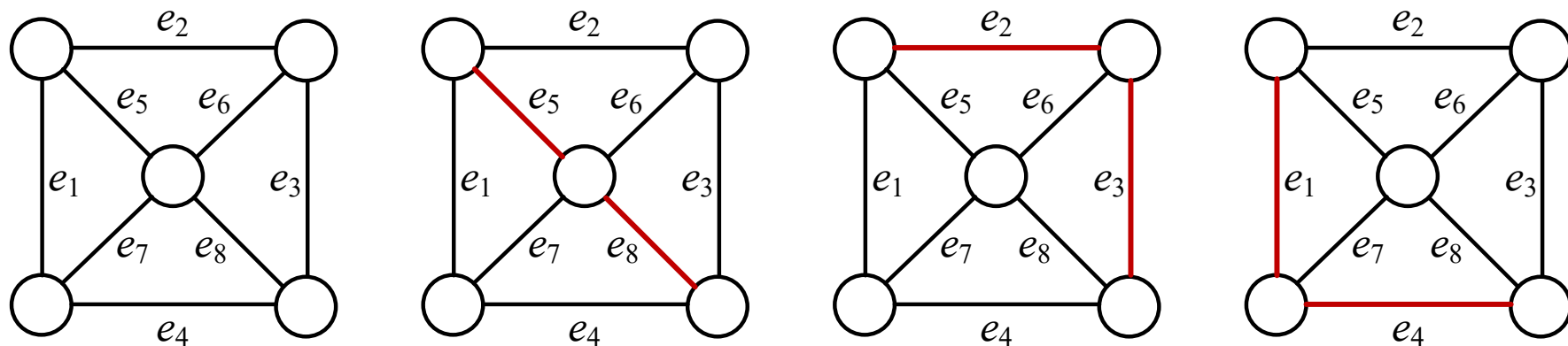


2.6. Conectividade

k -Conectividade

Um grafo $G = (V, E)$ é k -conexo se e somente se para todo para $v_i, v_j \in V, v_i \neq v_j$ existirem ao menos k caminhos disjuntos.

Dois caminhos entre os vértices v_i e v_j de um grafo são disjuntos se não possuírem arestas em comum.



Três caminhos disjuntos do grafo G constituídos respectivamente pelas arestas $\{e_5, e_8\}$, $\{e_2, e_3\}$ e $\{e_1, e_4\}$

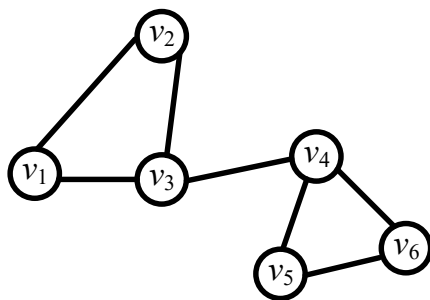


2.6. Conectividade

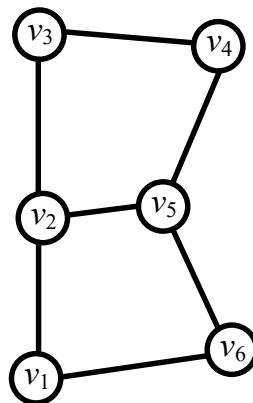
k -Conectividade

Um grafo $G = (V, E)$ é k -conexo se e somente se para todo para $v_i, v_j \in V, v_i \neq v_j$ existirem ao menos k caminhos disjuntos.

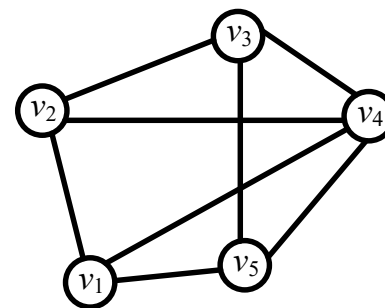
Dois caminhos entre os vértices v_i e v_j de um grafo são disjuntos se não possuírem arestas em comum.



1-conexo



2-conexo



3-conexo

Para todo grafo k -conexo ocorre as seguintes propriedades:

$\kappa(G) \leq \delta(G)$: conectividade de G é menor ou igual ao menor grau de um vértice de G .

$\kappa(G) \leq k$: conectividade de G é menor ou igual a quantidade k de caminhos disjuntos em G .

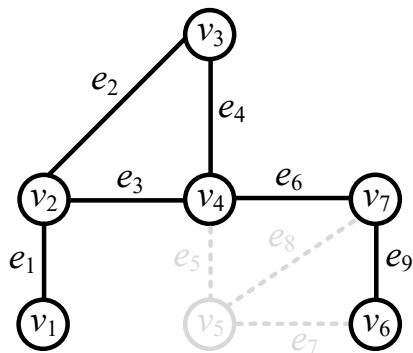
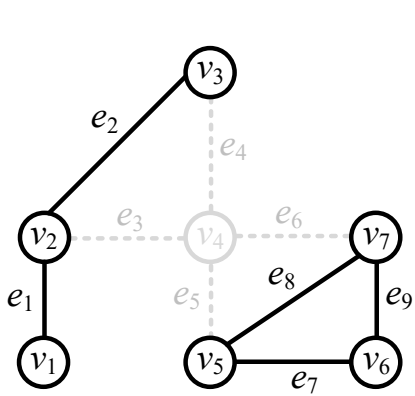
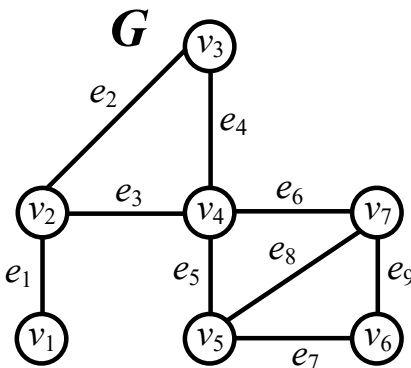


Articulação

Vértice de Articulação

Um vértice de articulação de um grafo G é **aquele cuja remoção resulta na desconexão de G** .

O vértice v_4 é de articulação, já o vértice v_5 não é pois não divide G em componentes.





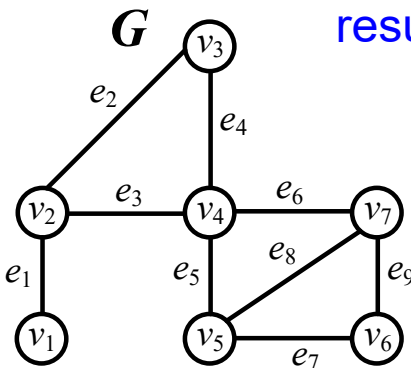
2.6. Conectividade

Articulação

Vértice de Articulação

Um vértice de articulação de um grafo G é aquele cuja remoção resulta na desconexão de G .

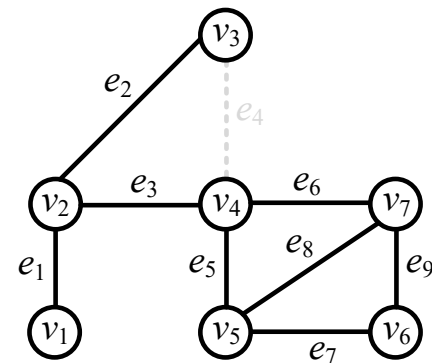
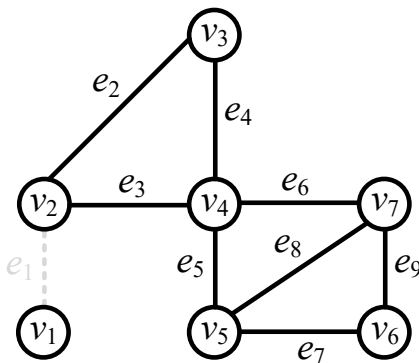
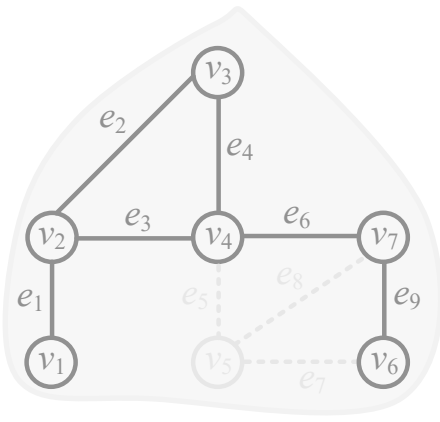
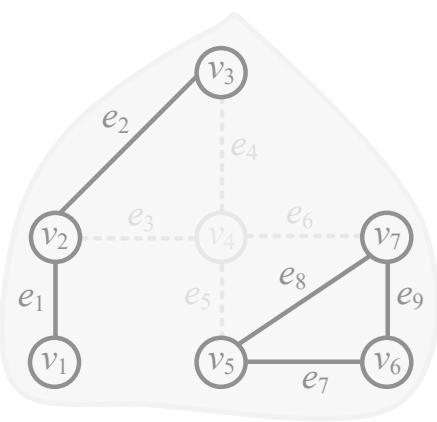
O vértice v_4 é de articulação, já o vértice v_5 não é pois não divide G em componentes.



Aresta de Articulação

Uma aresta de articulação de um grafo G é aquela cuja remoção resulta na desconexão de G .

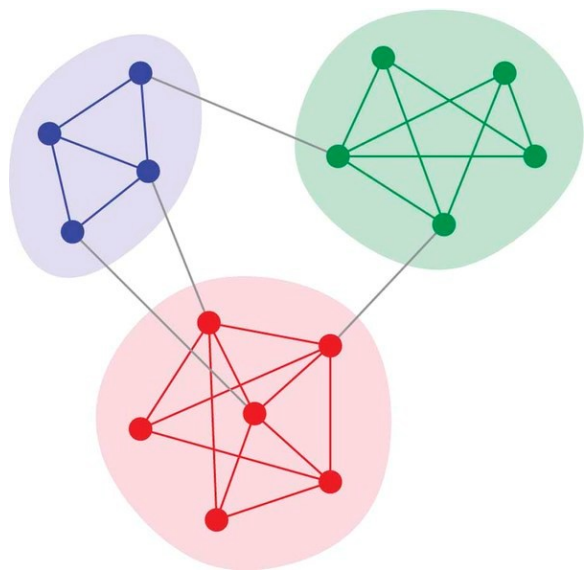
A aresta e_1 é de articulação, já a aresta e_4 não é.





Cliques

Em um grafo não-direcionado $G = (V, E)$ é um subconjunto de vértices $C \subseteq V$, tal que para todo par de vértices (v_i, v_j) e $v_i \neq v_j$, existe uma aresta incidente conectando v_i e v_j . Isso equivale a dizer que um **subgrafo induzido** de C é **completo**.



Clique maximal: cliques que não são subconjunto de outros subgrafos completos de G .



Clique máxima: é o maior clique possível de G . O **número do clique** $\omega(G)$ é o número de vértices de um clique máximo de G .



Perguntas? Sugestões?

